InventarioWeb El Jardín

Documento de Especificación y Diseño de Software (DEDS)

Fase 4 — Documento Final Integrado

Estudiante: Axarely Jahoska Escobar Jaime

Curso: Metodología y Diseño de Sistemas I — Cuatrimestre II 2026

Base bibliográfica: Ingeniería del Software: Un Enfoque Práctico — Roger S. Pressman, 7.ª edición

Estándar aplicado: IEEE 830-1998

Fecha de entrega: 10 de agosto de 2026

Versión del documento: 1.0 — Final

Control de versiones

Tabla 1 — Control de versiones del documento DEDS completo.

Versión	Fecha	Cambios	Autor
1.0	10/08/2026	Documento DEDS final: integración de las Fases I, II, III en un único documento, con tabla de contenidos, lista de figuras y tablas, y el nuevo Capítulo 4 de reflexiones.	A. Escobar

Tabla de Contenidos

Capítulo 1 — Marco del Proyecto 5
1.1 Descripción del problema 5
1.2 Caracterización del sistema de software 6
1.3 Stakeholders e interesados 7
1.4 Modelo de proceso adoptado y justificación 8
1.5 Plan de trabajo 9
1.6 Gestión de riesgos inicial 10
Capítulo 2 — Especificación de Requerimientos (SRS — IEEE 830) 11
2.1 Introducción 11
2.2 Descripción general del sistema 12
2.3 Requerimientos funcionales 13
2.4 Requerimientos no funcionales 15
2.5 Requerimientos de interfaz 16
2.6 Casos de uso 17
2.7 Diagrama de clases de análisis 20
2.8 Diagramas de actividades 20
2.9 Diagramas de secuencia 22
2.10 Diagrama de estados 24
Capítulo 3 — Diseño del Software 25
3.1 Diseño arquitectónico 26
3.2 Diseño de componentes 28
3.3 Diseño de la interfaz de usuario 32
3.4 Diseño basado en patrones 40
3.5 Diseño de la webapp 42
Capítulo 4 — Reflexiones y Lecciones Aprendidas 44
4.1 Decisiones técnicas clave 44
4.2 Dificultades encontradas y soluciones aplicadas 44
4.3 Aspectos a mejorar 45
4.4 Aprendizajes para la fase de desarrollo 45
Referencias Bibliográficas 47

Lista de Figuras y Lista de Tablas

Lista de Figuras

Figura 1 — Diagrama de casos de uso general de InventarioWeb El Jardín.

Figura 2 — Diagrama de clases de análisis de InventarioWeb El Jardín.

Figura 3 — Diagrama de actividades del proceso de registro de entrada de mercadería.

Figura 4 — Diagrama de actividades del proceso de registro de salida hacia sucursal.

Figura 5 — Diagrama de secuencia — Registrar entrada de mercadería (UC02).

Figura 6 — Diagrama de secuencia — Registrar salida hacia sucursal, incluyendo la generación de alerta (UC03 incluye UC05).

Figura 7 — Diagrama de secuencia — Consultar existencias por sucursal (UC04).

Figura 8 — Diagrama de estados de la entidad Existencia por sucursal/bodega.

Figura 9 — Diagrama de arquitectura en capas de InventarioWeb El Jardín.

Figura 10 — Diagrama de despliegue de InventarioWeb El Jardín.

Figura 11 — Diagrama de componentes de InventarioWeb El Jardín (v1.1, corregido).

Figura 12 — Diagrama de clases de diseño de InventarioWeb El Jardín (v1.1, métodos de Existencia unificados con el patrón Observer).

Figura 13 — Pantalla de inicio de sesión, común a los tres roles.

Figura 14 — Panel de indicadores comparativo entre sucursales, para los dueños (UC07).

Figura 15 — Registro de entrada de mercadería a bodega, con la existencia actual visible como contexto (UC02).

Figura 16 — Registro de salida hacia sucursal, con validación de existencia y aviso de excepción (UC03).

Figura 17 — Consulta de existencias por sucursal, filtrable y con estado visual por producto (UC04).

Figura 18 — Listado de alertas de bajo inventario pendientes (UC05).

Figura 19 — Gestión de proveedores (CRUD básico) — pantalla nueva que cierra el pendiente de la Fase 2.

Figura 20 — Prototipo de navegación de InventarioWeb El Jardín por rol de usuario.

Figura 21 — Guía de estilo de InventarioWeb El Jardín: paleta de colores, tipografía y componentes base.

Figura 22 — Aplicación del patrón Factory Method a la creación de movimientos de inventario.

Figura 23 — Aplicación del patrón Observer a la notificación de cambios de existencia.

Figura 24 — Aplicación del patrón Adapter a la integración de proveedores externos de notificación.

Figura 25 — Mapa de contenido de InventarioWeb El Jardín (v1.1, con Gestionar proveedores).

Lista de Tablas

Tabla 1 — Control de versiones del documento DEDS completo.

Tabla 2 — Control de versiones del Capítulo 1.

Tabla 3 — Plan de trabajo del proyecto por incremento.

Tabla 4 — Riesgos del proyecto y estrategia de mitigación.

Tabla 5 — Control de versiones del Capítulo 2.

Tabla 6 — Requerimientos funcionales de InventarioWeb El Jardín.
Tabla 7 — Requerimientos no funcionales de InventarioWeb El Jardín.
Tabla 8 — Control de versiones del Capítulo 3.

Capítulo 1 — Marco del Proyecto

Este capítulo corresponde a la Fase 1 del proyecto (versión 1.1, corregida según la retroalimentación del profesor), y establece el contexto completo de InventarioWeb El Jardín: qué se va a construir, por qué es un problema de ingeniería de software y cómo se gestiona metodológicamente.

Tabla 2 — Control de versiones del Capítulo 1.

Versión	Fecha	Cambios	Autor
1.0	03/07/2026	Versión inicial del documento. Entrega de la Fase 1: marco del proyecto, contexto y selección del modelo de proceso.	A. Escobar
1.1	28/07/2026	Corrección según retroalimentación del profesor: se agrega la sección de Plan de Trabajo, se restaura la tabla de control de versiones, se amplía el análisis comparativo de modelos de proceso incluyendo el modelo en espiral, y se agrega el argumento de escalabilidad por sucursal en la justificación del modelo adoptado.	A. Escobar

1.1 Descripción del problema

El restaurante El Jardín abastece sus dos sucursales, Santa Cruz y Peña Blanca, desde una única bodega central, pero no cuenta con un mecanismo digital que separe ni controle el inventario de forma independiente por sucursal. Actualmente el registro de entradas y salidas se realiza en una hoja de cálculo de Excel y los reportes visuales se generan de manera desconectada del proceso operativo diario, lo que provoca varios problemas recurrentes:

- Dificultad para determinar con precisión qué consumió cada sucursal en un período dado.
- Faltantes o sobrantes de producto que no se detectan a tiempo.
- Reposición de insumos basada en estimaciones del bodeguero y no en datos reales y actualizados.

- Imposibilidad de analizar tendencias de consumo por sucursal o por producto de forma ágil.
- Registros incompletos, fuera de tiempo o con diferencias entre lo entregado y lo recibido, sin trazabilidad clara del producto.

La gerencia del restaurante requiere una plataforma digital moderna, accesible para los tres roles involucrados en el proceso, que permita separar el inventario de cada sucursal, mantener la información sincronizada en todo momento y visualizar el estado del inventario sin depender de reportes generados manualmente.

InventarioWeb El Jardín es la aplicación web que resuelve este problema: registra en línea las entradas de mercadería a la bodega y las salidas hacia cada sucursal, calcula automáticamente las existencias disponibles por producto y por punto de venta, y ofrece paneles visuales de consumo, comparativos entre sucursales y alertas de bajo inventario. Los roles de usuario (dueños, bodeguero y encargados de sucursal) tienen permisos diferenciados según sus responsabilidades.

1.2 Caracterización del sistema de software

Pressman (cap. 1) agrupa el software de computadora en siete grandes dominios de aplicación: software de sistemas, software de aplicación, software de ingeniería y ciencias, software incrustado, software de línea de productos, aplicaciones web (webapps) y software de inteligencia artificial. InventarioWeb El Jardín se ubica en el dominio de aplicaciones web (webapps): un software centrado en redes, accesible mediante navegador, cuya función es capturar y presentar información a distintos usuarios conectados a través de una red.

Dentro de esa categoría, el sistema muestra atributos propios de una webapp según Pressman: es orientado a datos, requiere concurrencia (bodeguero y encargados de ambas sucursales acceden simultáneamente), depende de disponibilidad durante el horario operativo, y exige atención a la seguridad (cada rol accede únicamente a la información que le corresponde). Se trata de un nuevo desarrollo: aunque el restaurante ya tiene un proceso de control de inventario apoyado en Excel y en una herramienta de análisis de escritorio, no existe una aplicación de software previa que automatice el registro y el cálculo de existencias.

Características de calidad prioritarias

- Usabilidad: el bodeguero y los encargados de sucursal no tienen un perfil técnico, por lo que la interfaz debe ser simple e intuitiva.
- Fiabilidad: los cálculos de existencias deben ser consistentes y reflejar con exactitud los movimientos reales registrados.
- Disponibilidad: el sistema debe estar accesible durante el horario operativo de bodega y sucursales.
- Seguridad: el acceso debe diferenciarse por rol, de modo que cada usuario solo pueda ver o modificar la información que le corresponde.
- Mantenibilidad: el sistema debe poder ajustarse a futuro (nuevos productos, nuevas sucursales) sin rediseñar la solución completa.

Principales desafíos de ingeniería

- Diseñar un modelo de datos que separe correctamente el inventario por sucursal, evitando el problema actual de una cuenta única sin diferenciación.
- Garantizar que los movimientos registrados por el bodeguero se reflejen de forma consistente en las existencias visibles para cada sucursal.
- Construir visualizaciones e indicadores dentro de la propia aplicación web, sin depender de una herramienta externa de escritorio.
- Definir un esquema de roles y permisos que sea sencillo de administrar, pero suficientemente seguro.
- Lograr la adopción del sistema por parte de usuarios acostumbrados a un proceso manual, minimizando la resistencia al cambio.

1.3 Stakeholders e interesados

- Dueños del restaurante: consultan reportes e indicadores para la toma de decisiones de compra y de negocio. Interesados directos en el retorno de la inversión del sistema.

- Bodeguero: registra las entradas de mercadería a la bodega y las salidas hacia cada sucursal. Usuario con mayor frecuencia de uso.
- Encargados de sucursal: consultan las existencias disponibles en su sucursal y reportan el consumo o las ventas.
- Proveedor: actor externo que entrega la materia prima a la bodega; no interactúa directamente con el sistema, pero sus entregas originan los registros de entrada.
- Estudiante (analista/desarrollador): responsable del análisis, diseño e implementación del sistema en el III Cuatrimestre 2026.

1.4 Modelo de proceso adoptado y justificación

Se evaluaron tres modelos de proceso de software estudiados en el curso: el modelo en cascada, el modelo en espiral y el modelo incremental con prácticas ágiles de Scrum adaptado.

- Modelo en cascada: organiza el trabajo en fases secuenciales y estrictas, donde cada fase concluye por completo antes de iniciar la siguiente. Exige que los requerimientos estén completamente definidos desde el inicio, algo poco realista para un proceso de negocio que aún no ha sido formalizado ni documentado en detalle.
- Modelo en espiral: combina la naturaleza iterativa de la construcción de prototipos con los aspectos controlados del modelo de cascada, incorporando análisis de riesgos explícito en cada vuelta. Es potente para proyectos grandes y de alto riesgo, pero exige un esfuerzo de gestión de riesgos formal desproporcionado para un proyecto académico de un solo desarrollador y ocho semanas de calendario fijo.
- Modelo incremental con prácticas ágiles de Scrum adaptado: el primer incremento entrega el producto núcleo y los siguientes lo refinan con base en la retroalimentación recibida. Permite ajustar el modelo de datos y los requerimientos conforme se profundiza en el proceso real de bodega y sucursales.

Se adopta el modelo incremental con prácticas ágiles de Scrum adaptado. El proceso de inventario del restaurante nunca ha sido documentado formalmente, por lo que el entendimiento del problema se refina a medida que avanza la elicitación; un modelo en cascada no permitiría

incorporar esos ajustes sin retrasar todo el proyecto, y un modelo en espiral impondría una carga de gestión de riesgos que excede el alcance de un proyecto académico individual. El cronograma del curso, organizado en cuatro incrementos de duración fija, se ajusta de forma natural a este ciclo.

El modelo incremental encaja además con la propia estructura del negocio: InventarioWeb El Jardín puede construirse comenzando con un núcleo que cubra la bodega central y una sola sucursal (Santa Cruz), y agregar la segunda sucursal (Peña Blanca) como un incremento posterior una vez validado el modelo de datos. Esta forma de crecer por sucursal es exactamente el tipo de escalabilidad que el modelo incremental está diseñado para soportar, y facilita además la continuidad hacia el III Cuatrimestre, cuando el mismo proyecto continuará bajo la misma metodología ágil.

1.5 Plan de trabajo

El plan de trabajo sigue los cuatro incrementos definidos por el curso, cada uno con sus actividades principales y su entregable documentado:

Tabla 3 — Plan de trabajo del proyecto por incremento.

Incremento	Semanas / Entrega	Actividades principales	Entregable
Fase 1	Semanas 1–2 · 3 jul 2026	Levantamiento inicial del problema con dueños y bodeguero. Caracterización del software según Pressman. Comparación y selección del modelo de proceso.	Marco del proyecto (Cap. 1 del DEDS)
Fase 2	Semanas 3–5 · 17 jul 2026	Entrevistas de elicitación con bodeguero y encargados de sucursal. Redacción del SRS bajo IEEE 830. Modelado UML (casos de uso, clases, secuencia, actividades, estados).	SRS completo + diagramas UML
Fase 3	Semanas 6–7 · 31 jul 2026	Diseño de la arquitectura y de los componentes del sistema. Diseño de wireframes y navegación. Aplicación de patrones de diseño.	Documento de diseño del software
Fase 4	Semana 8 · 7–10 ago 2026	Integración de las tres fases en un único DEDS. Incorporación de retroalimentación acumulada.	DEDS final + presentación oral

Incremento	Semanas / Entrega	Actividades principales	Entregable
		Preparación y ensayo de la presentación oral.	

Este plan se articula directamente con la metodología incremental adoptada: cada fila corresponde a un incremento cerrado y evaluado antes de iniciar el siguiente, de modo que la retroalimentación del profesor entre fases cumple la misma función que la revisión de sprint en Scrum.

1.6 Gestión de riesgos inicial

A continuación se identifican los principales riesgos detectados al inicio del proyecto y la estrategia prevista para mitigarlos:

Tabla 4 — Riesgos del proyecto y estrategia de mitigación.

Riesgo	Estrategia de mitigación	Impacto
Resistencia al cambio del bodeguero y encargados de sucursal, acostumbrados al proceso manual en Excel.	Diseñar una interfaz simple, involucrar a los usuarios desde la elicitación de requerimientos y capacitarlos antes de la puesta en marcha.	Alto
Datos históricos de inventario incompletos o inconsistentes en el archivo actual de Excel.	Validar y depurar una muestra de los datos existentes antes de definir el modelo de datos final en la Fase 2.	Medio
Disponibilidad limitada del personal del restaurante para sesiones de elicitación durante el horario operativo.	Programar entrevistas cortas fuera de las horas pico y aprovechar observación directa del proceso.	Medio
Alcance mal definido del modelo de datos por sucursal, heredado de la ambigüedad del proceso actual.	Revisar y validar el modelo de datos con la gerencia al cierre de cada incremento, antes de avanzar a la siguiente fase.	Alto

Capítulo 2 — Especificación de Requerimientos (SRS — IEEE 830)

Este capítulo corresponde a la Fase 2 del proyecto: la Especificación de Requerimientos de Software de InventarioWeb El Jardín, elaborada conforme al estándar IEEE 830-1998.

Tabla 5 — Control de versiones del Capítulo 2.

Versión	Fecha	Cambios	Autor
1.0	17/07/2026	Versión inicial de la Fase 2: especificación de requerimientos (SRS, IEEE 830) y modelado UML (casos de uso, clases, actividades, secuencia y estados).	A. Escobar

2.1 Introducción

2.1.1 Propósito

Este documento describe de forma completa y sin ambigüedades qué debe hacer el sistema, para que sirva como base de acuerdo entre el estudiante (en su doble papel de analista y desarrollador) y la gerencia del restaurante El Jardín (en su papel de cliente), y como insumo directo para el diseño (Capítulo 3) y el desarrollo del III Cuatrimestre 2026.

2.1.2 Ámbito del sistema

InventarioWeb El Jardín SÍ cubre: el registro de entradas y salidas de mercadería, el cálculo automático de existencias por ubicación, la generación de alertas de bajo inventario, la consulta de indicadores comparativos entre sucursales, y la gestión de usuarios, roles y catálogo de productos. NO cubre en esta etapa: facturación o punto de venta, contabilidad financiera, gestión de nómina, ni el despliegue en un ambiente de producción con usuarios reales.

2.1.3 Definiciones, siglas y abreviaturas

- SRS: Especificación de Requerimientos de Software.
- RF / RNF: Requerimiento Funcional / Requerimiento No Funcional.
- UML: Lenguaje Unificado de Modelado.
- Existencia: cantidad disponible de un producto en una ubicación específica en un momento dado.

- Stock mínimo (umbral): cantidad de un producto por debajo de la cual el sistema genera una alerta.
- RBAC: control de acceso basado en roles.

2.1.4 Referencias

- Pressman, R. S. (2010). Ingeniería del Software: Un Enfoque Práctico (7.ª ed.). McGraw-Hill.
- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications.
- InventarioWeb El Jardín — Capítulo 1 de este mismo documento (Fase 1, versión 1.1).

2.2 Descripción general del sistema

InventarioWeb El Jardín es un producto nuevo e independiente que digitaliza un proceso hoy manual. Operará como una aplicación web autocontenida entre la bodega central y las dos sucursales (Santa Cruz y Peña Blanca).

Funciones del producto

- Registrar el ingreso de mercadería a la bodega central y el envío hacia cada sucursal.
- Calcular y mantener actualizadas las existencias por bodega y por sucursal de forma independiente.
- Generar alertas automáticas de bajo inventario.
- Presentar paneles de indicadores y reportes comparativos entre sucursales.
- Administrar el catálogo de productos, los proveedores y los usuarios del sistema.

Características de los usuarios

- Dueños: perfil gerencial, no técnico, con experiencia previa consultando reportes visuales.
- Bodeguero: perfil operativo, sin formación técnica; mayor frecuencia de uso del sistema.
- Encargados de sucursal: perfil operativo, sin formación técnica.

Restricciones y suposiciones

- El sistema debe ser accesible únicamente mediante navegador web, sin instalación de software adicional.
- El proyecto debe completarse dentro del calendario académico de 8 semanas del curso.
- Se asume conexión a internet estable en bodega y ambas sucursales durante el horario operativo.
- Se asume que la gerencia proporcionará el catálogo inicial de productos y los valores de stock mínimo.

2.3 Requerimientos funcionales

A continuación se detallan los requerimientos funcionales (RF) identificados, cada uno con su prioridad y su criterio de aceptación:

Tabla 6 — Requerimientos funcionales de InventarioWeb El Jardín.

ID	Descripción	Prioridad	Criterio de aceptación
RF-01	El sistema debe autenticar usuarios y otorgar acceso según su rol (dueño, bodeguero, encargado de sucursal).	Alta	Un usuario con credenciales válidas accede únicamente a las funciones permitidas para su rol; con credenciales inválidas, el acceso es rechazado.
RF-02	El sistema debe permitir registrar la entrada de mercadería a la bodega central, indicando producto, cantidad y proveedor.	Alta	Tras registrar una entrada válida, la existencia de la bodega aumenta exactamente en la cantidad ingresada.
RF-03	El sistema debe permitir registrar la salida de mercadería desde la bodega hacia una sucursal específica.	Alta	Tras registrar una salida válida, la existencia de la bodega disminuye y la de la sucursal destino aumenta en la cantidad enviada.
RF-04	El sistema debe calcular y mantener actualizadas las existencias de cada producto de forma independiente por bodega y por sucursal.	Alta	En todo momento, la existencia mostrada para una ubicación refleja la suma de sus movimientos de entrada y salida registrados.
RF-05	El sistema debe permitir consultar las existencias disponibles de una sucursal específica.	Alta	El encargado de una sucursal visualiza únicamente las

ID	Descripción	Prioridad	Criterio de aceptación
			existencias correspondientes a su sucursal.
RF-06	El sistema debe generar una alerta automática cuando la existencia de un producto en una ubicación cae por debajo de su stock mínimo.	Alta	Al registrarse un movimiento que deja la existencia bajo el umbral, se crea una alerta visible para los dueños en un plazo no mayor a la actualización inmediata de la existencia.
RF-07	El sistema debe permitir consultar indicadores de consumo por sucursal y por producto en un período determinado.	Media	El usuario puede filtrar por sucursal, producto y rango de fechas, y obtener el consumo total correspondiente.
RF-08	El sistema debe presentar un panel comparativo del consumo entre las sucursales.	Media	El panel muestra, para un mismo período, el consumo de Santa Cruz y de Peña Blanca lado a lado.
RF-09	El sistema debe permitir gestionar el catálogo de productos (crear, editar y desactivar productos).	Media	Un producto creado o editado queda disponible para su uso inmediato en los formularios de entrada y salida.
RF-10	El sistema debe permitir gestionar usuarios y asignarles un rol del sistema.	Media	Un usuario nuevo puede iniciar sesión y acceder únicamente a las funciones de su rol asignado.
RF-11	El sistema debe permitir asociar un proveedor a cada movimiento de entrada de mercadería.	Media	Todo registro de entrada exitoso queda vinculado a un proveedor existente en el sistema.
RF-12	El sistema debe permitir consultar el historial de movimientos de un producto o de una ubicación.	Media	El usuario obtiene una lista cronológica de movimientos con fecha, tipo, cantidad y usuario que los registró.
RF-13	El sistema debe permitir reportar discrepancias entre la cantidad entregada por bodega y la recibida en sucursal.	Alta	Una discrepancia reportada queda registrada y asociada al movimiento correspondiente para su trazabilidad.
RF-14	El sistema debe permitir exportar reportes de inventario en formato PDF o Excel.	Baja	El archivo exportado contiene la misma información mostrada en pantalla al momento de la exportación.

ID	Descripción	Prioridad	Criterio de aceptación
RF-15	El sistema debe permitir editar o anular un movimiento registrado por error, exigiendo una justificación.	Media	Un movimiento anulado revierte su efecto sobre las existencias afectadas y queda marcado como anulado en el historial, sin eliminarse.
RF-16	El sistema debe notificar a los dueños cuando una sucursal alcanza un nivel crítico de inventario.	Alta	Ante una existencia en estado "Agotado", los dueños reciben una notificación además de la alerta visible en el panel.
RF-17	El sistema debe presentar un panel (dashboard) de indicadores en tiempo real para los dueños.	Media	Los indicadores mostrados reflejan los movimientos registrados hasta el momento de la consulta, sin necesidad de actualización manual.
RF-18	El sistema debe permitir a cualquier usuario cerrar su sesión de forma segura.	Baja	Tras cerrar sesión, el usuario no puede acceder a funciones protegidas sin autenticarse nuevamente.

2.4 Requerimientos no funcionales

Estos requerimientos se derivan directamente de las características de calidad prioritarias identificadas en el Capítulo 1:

Tabla 7 — Requerimientos no funcionales de InventarioWeb El Jardín.

ID	Categoría	Descripción
RNF-01	Rendimiento	Las consultas de existencias y los paneles de indicadores deben responder en menos de 2 segundos bajo condiciones normales de operación.
RNF-02	Seguridad	Las contraseñas deben almacenarse con una función de hash; el acceso a las funciones debe controlarse por rol (RBAC); toda comunicación debe realizarse mediante HTTPS.
RNF-03	Usabilidad	Las tareas más frecuentes (registrar entrada, registrar salida, consultar existencias) deben completarse en un máximo de 3 pasos, sin requerir capacitación técnica extensa.
RNF-04	Disponibilidad	El sistema debe estar disponible al menos el 95% del horario operativo del restaurante.
RNF-05	Mantenibilidad	La arquitectura debe permitir agregar nuevas sucursales o productos sin necesidad de rediseñar los módulos existentes.

ID	Categoría	Descripción
RNF-06	Escalabilidad	El sistema debe soportar el acceso concurrente del bodeguero y de los encargados de ambas sucursales sin degradación perceptible del servicio.
RNF-07	Portabilidad	El sistema debe funcionar correctamente en los navegadores modernos más utilizados (Chrome, Edge, Firefox) sin instalación adicional.
RNF-08	Confiabilidad	Los movimientos de inventario deben registrarse de forma transaccional, de modo que una falla durante el proceso no deje existencias en un estado inconsistente.

2.5 Requerimientos de interfaz

Interfaces de usuario

Interfaz web responsiva, utilizable tanto desde una computadora de escritorio en bodega como desde un dispositivo móvil en sucursal.

Interfaces de hardware

No se requiere hardware especializado: basta con cualquier dispositivo con navegador web y conexión a internet.

Interfaces de software

Motor de base de datos relacional para productos, existencias, movimientos y usuarios. Integración opcional con un servicio externo de notificaciones (correo o SMS).

Interfaces de comunicación

Comunicación entre frontend y backend mediante HTTPS, a través de una API de tipo REST.

2.6 Casos de uso

El diagrama general agrupa los casos de uso identificados para los tres actores principales del sistema. El proveedor es un actor externo que origina las entradas de mercadería pero no interactúa directamente con el sistema, por lo que no se modela como actor en este diagrama.

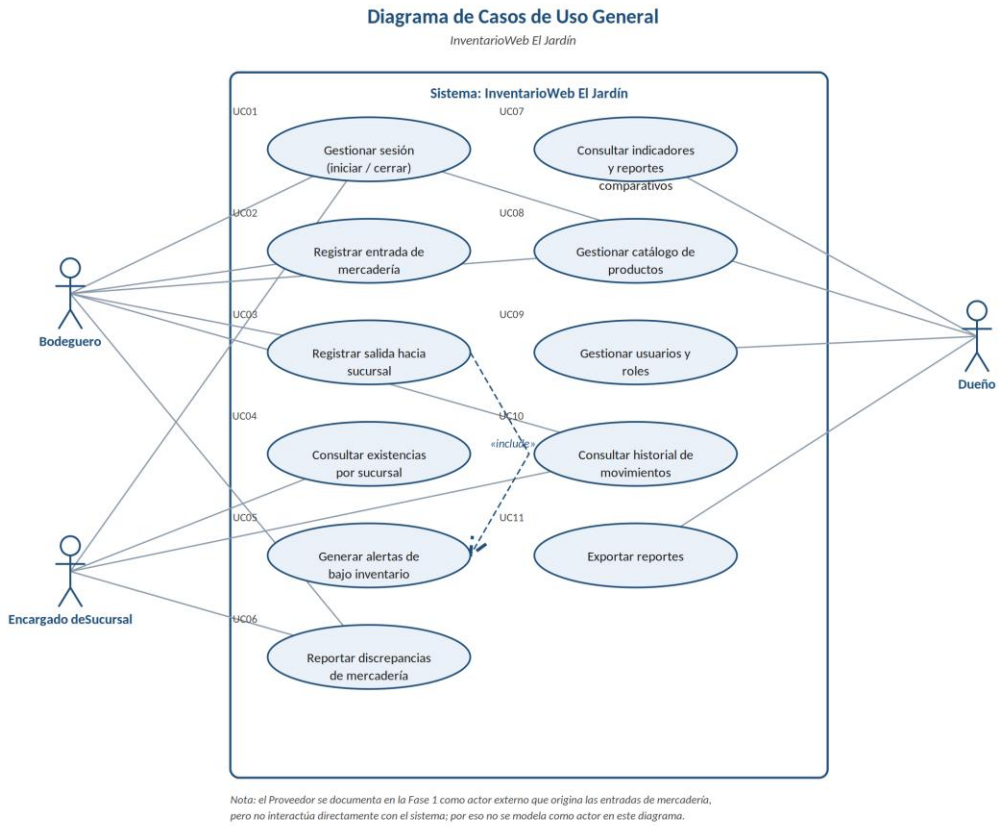


Figura 1 — Diagrama de casos de uso general de InventarioWeb El Jardín.

2.6.1 Ficha detallada — UC02: Registrar entrada de mercadería a bodega

Actor principal	Bodeguero
Precondiciones	El bodeguero ha iniciado sesión con su rol. El producto recibido existe en el catálogo del sistema.
Flujo principal	1. El bodeguero selecciona la opción "Registrar entrada". 2. El sistema presenta el formulario de entrada. 3. El bodeguero selecciona el producto, ingresa la cantidad recibida y el proveedor. 4. El sistema valida los datos ingresados. 5. El sistema registra el movimiento de tipo entrada. 6. El sistema actualiza la existencia de la bodega sumando la cantidad ingresada. 7. El sistema confirma el registro al bodeguero.
Flujo alternativo	A1 — Producto no existe en catálogo: el sistema permite crearlo antes de continuar.
Excepciones	E1 — Cantidad no numérica o negativa: el sistema rechaza el registro. E2 — Falla de conexión con la base de datos: el sistema informa el error sin aplicar cambios parciales.

Postcondiciones	La existencia de la bodega queda actualizada y el movimiento queda registrado en el historial.

2.6.2 Ficha detallada — UC03: Registrar salida de mercadería hacia sucursal

Actor principal	Bodeguero
Flujo principal	1. El bodeguero selecciona "Registrar salida". 2. Selecciona la sucursal destino y el producto con su cantidad. 3. El sistema valida que la bodega tenga existencia suficiente. 4. El sistema registra el movimiento de tipo salida. 5. El sistema actualiza la existencia de bodega (resta) y de sucursal (suma). 6. El sistema evalúa si la existencia resultante quedó bajo el stock mínimo e invoca «include» el caso de uso UC05 (Generar alertas de bajo inventario) cuando corresponde. 7. El sistema confirma la salida al bodeguero.
Flujo alternativo	A1 — Envío parcial: si la cantidad solicitada supera la existencia disponible, el sistema sugiere la cantidad máxima disponible.
Excepciones	E1 — Existencia insuficiente y el bodeguero no ajusta la cantidad: el sistema rechaza el registro.
Postcondiciones	Existencias actualizadas en bodega y sucursal; alerta generada si corresponde (relación «include» con UC05, ver Figura 1).

2.6.3 Ficha detallada — UC04: Consultar existencias por sucursal

Actor principal	Encargado de sucursal
Flujo principal	1. El usuario selecciona "Consultar existencias". 2. El sistema determina la sucursal asociada al usuario. 3. El sistema consulta las existencias registradas para esa sucursal. 4. El sistema presenta la lista de productos con su cantidad actual y su estado.
Postcondiciones	Ninguna; es un caso de uso de solo consulta.

2.6.4 Ficha detallada — UC05: Generar alertas de bajo inventario

Actor principal	Sistema (caso de uso automático, incluido desde UC03)
Precondiciones	Un movimiento de salida deja la existencia de un producto por debajo de su stock mínimo.
Flujo principal	1. El sistema detecta que una existencia transicionó al estado "Stock Bajo" o "Agotado" (ver diagrama de estados, Figura 8). 2. El sistema crea un registro de alerta asociado a la existencia y a la ubicación. 3. El sistema notifica a los dueños (RF-16).

	4. Un dueño puede marcar la alerta como atendida una vez gestionada la reposición.
Postcondiciones	Alerta registrada y visible para los dueños hasta que sea marcada como atendida.

2.6.5 Ficha detallada — UC07: Consultar indicadores y reportes comparativos

Actor principal	Dueños
Flujo principal	1. El dueño accede al panel de indicadores. 2. El sistema consulta los movimientos y existencias de la bodega y de ambas sucursales. 3. El sistema calcula los indicadores correspondientes. 4. El sistema presenta el panel comparativo con gráficos e indicadores numéricos.
Postcondiciones	Ninguna; es un caso de uso de solo consulta.

2.7 Diagrama de clases de análisis

El diagrama identifica las clases de análisis del sistema y sus relaciones, incluyendo la generalización de Ubicación en Bodega y Sucursal.

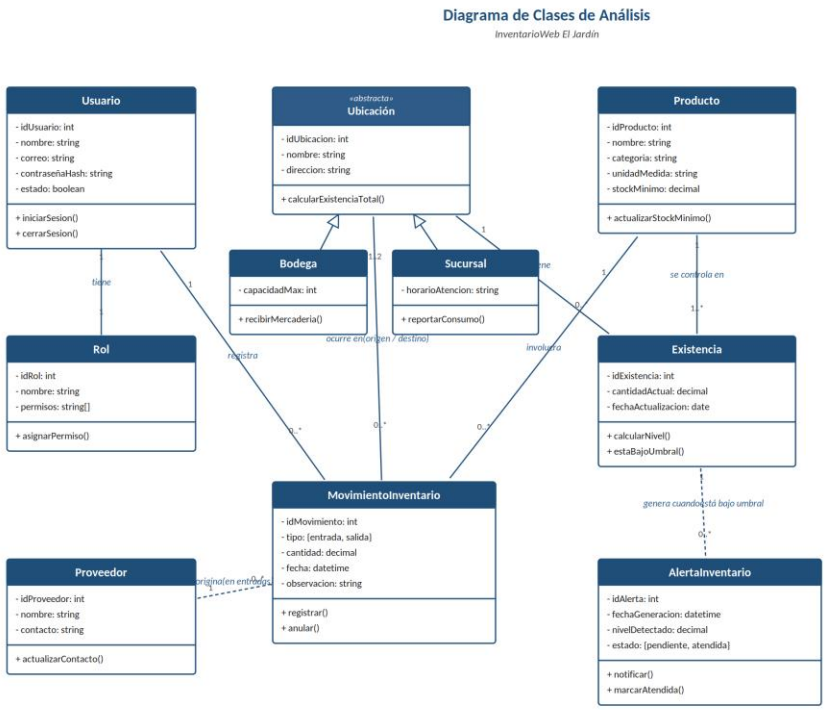


Figura 2 — Diagrama de clases de análisis de InventarioWeb El Jardín.

2.8 Diagramas de actividades

Se modelan los dos procesos críticos: el registro de entrada de mercadería y el registro de salida hacia una sucursal, este último con la lógica de decisión que determina si corresponde generar una alerta.

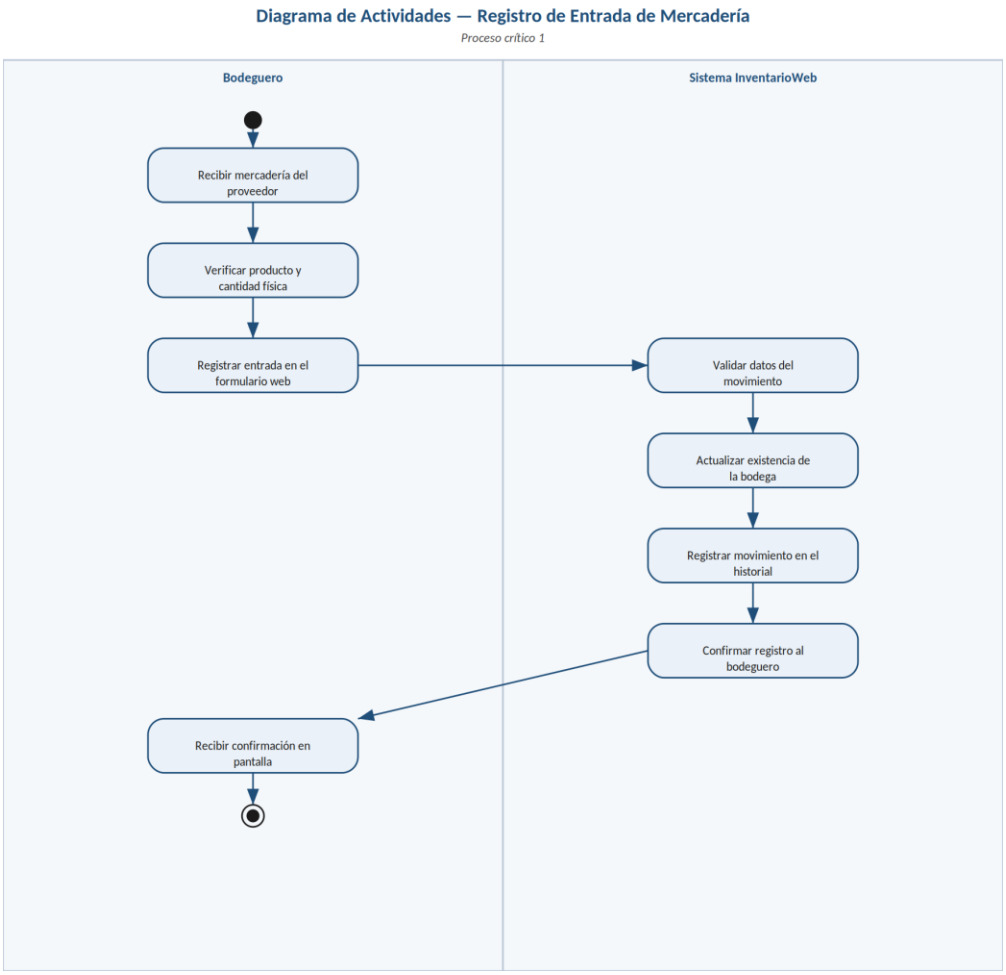


Figura 3 — Diagrama de actividades del proceso de registro de entrada de mercadería.

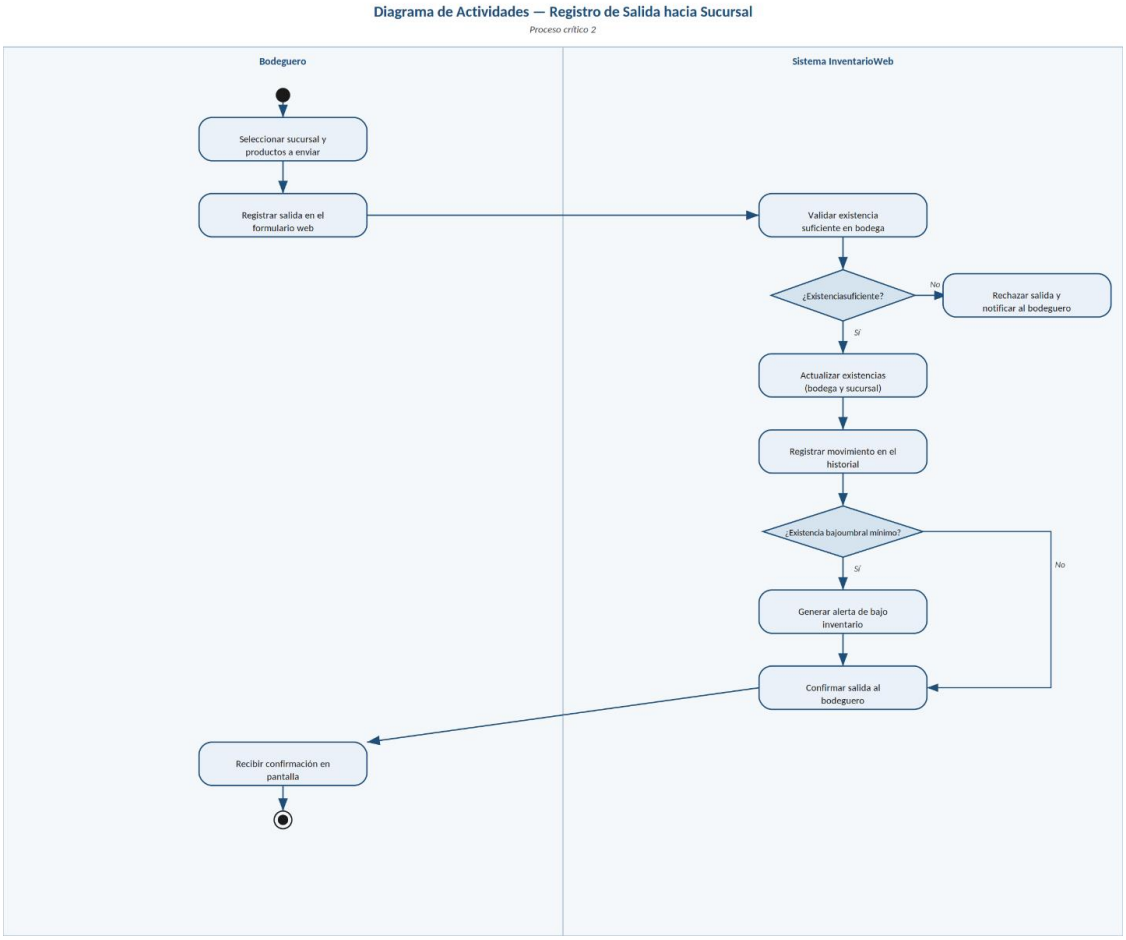


Figura 4 — Diagrama de actividades del proceso de registro de salida hacia sucursal.

2.9 Diagramas de secuencia

Los siguientes diagramas detallan la interacción entre actores y componentes internos para los tres casos de uso más representativos del corazón del sistema.

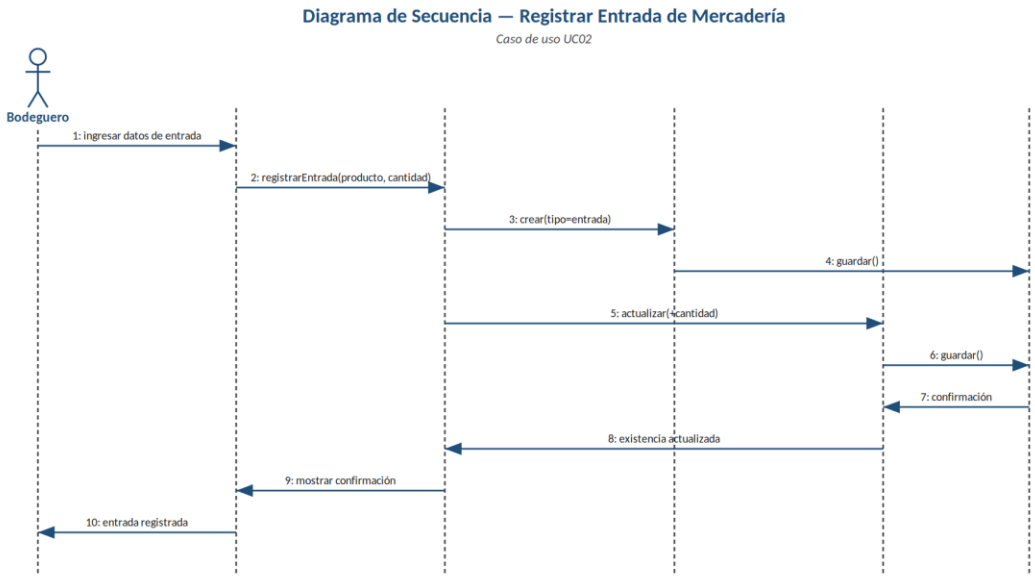


Figura 5 — Diagrama de secuencia — Registrar entrada de mercadería (UC02).

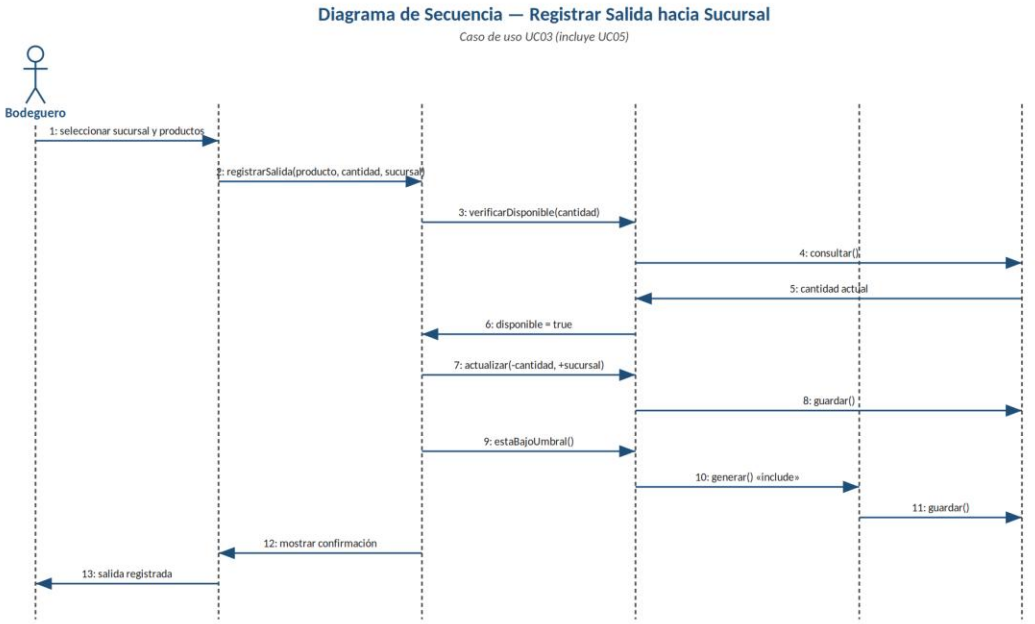


Figura 6 — Diagrama de secuencia — Registrar salida hacia sucursal, incluyendo la generación de alerta (UC03 incluye UC05).

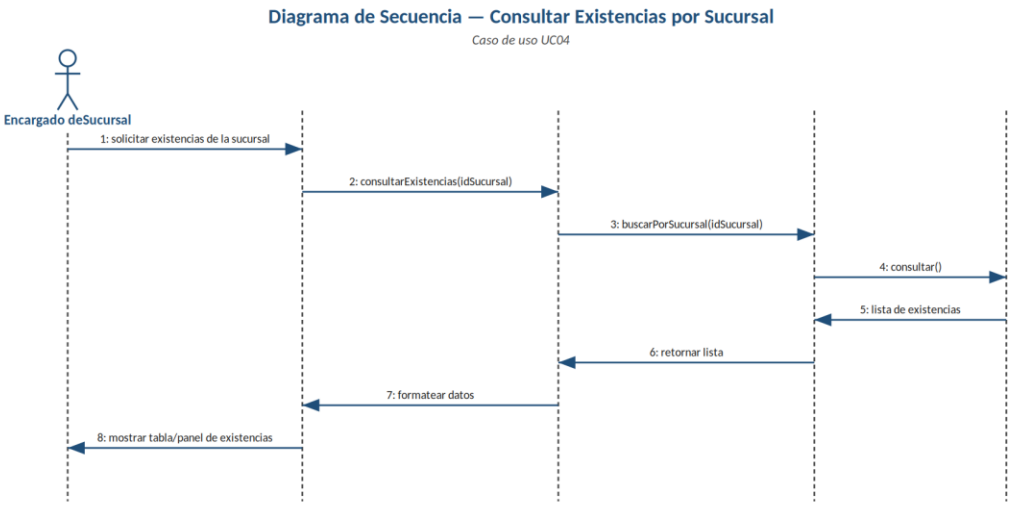


Figura 7 — Diagrama de secuencia — Consultar existencias por sucursal (UC04).

2.10 Diagrama de estados

Se modela el ciclo de vida de la entidad Existencia, disparador directo del caso de uso Generar alertas de bajo inventario.

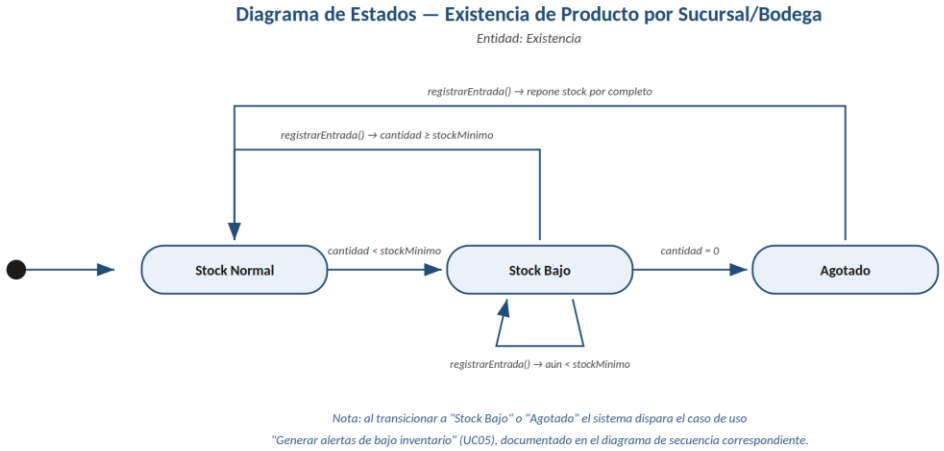


Figura 8 — Diagrama de estados de la entidad Existencia por sucursal/bodega.

Nota de cierre de pendientes (verificada para la Fase 4): RF-06 y RF-16 están completamente especificados (secciones 2.3 y fichas UC05); la relación «include» entre UC03 (Registrar salida) y UC05 (Generar alertas) está representada en el diagrama de casos de uso (Figura 1) y en el diagrama de secuencia de salida (Figura 6); y la transición Agotado → Stock Normal está representada en el diagrama de estados (Figura 8, transición

"registrarEntrada() → repone stock por completo"). Estos tres puntos, señalados como pendientes de confirmación en la retroalimentación de la Fase 3, quedan así cerrados.

Capítulo 3 — Diseño del Software

Este capítulo corresponde a la Fase 3 del proyecto (versión 1.1, corregida según la retroalimentación del profesor). Transforma la especificación de requerimientos del Capítulo 2 en una arquitectura y diseño detallado que guía la codificación del III Cuatrimestre 2026.

Tabla 8 — Control de versiones del Capítulo 3.

Versión	Fecha	Cambios	Autor
1.0	31/07/2026	Versión inicial de la Fase 3: diseño arquitectónico, de componentes, de interfaz de usuario, basado en patrones y de la webapp.	A. Escobar
1.1	13/08/2026	Corrección según retroalimentación del profesor: (1) el diagrama de componentes ya no muestra una relación «usa» directa entre Inventario y Alertas/Reportes, sino la notificación vía patrón Observer, consistente con el texto de la sección 3.3.4; (2) se conecta el Módulo Reportes al Panel de Indicadores; (3) se agrega el Módulo de Proveedores (componente, interfaz IProveedores, pantalla CRUD y entrada en el sitemap), cerrando el pendiente abierto desde la Fase 2; (4) se unifican los métodos de la clase Existencia entre el diagrama de clases de diseño y el diagrama del patrón Observer.	A. Escobar

3.1 Diseño arquitectónico

Estilo arquitectónico y justificación

Se adopta un estilo arquitectónico cliente-servidor de tres capas (presentación, lógica de negocio y datos), organizado internamente con el patrón MVC — en su variante MVT propia de Django. Se descarta una arquitectura de microservicios por ser desproporcionada para el volumen de este sistema y por multiplicar la complejidad operativa que un solo desarrollador tendría que sostener una vez entregado el sistema al negocio. Un estilo monolítico en capas se despliega como una sola unidad, es más simple de depurar y de mantener, y sigue siendo capaz de escalar verticalmente si el restaurante abre una tercera sucursal.

La separación en capas refleja directamente los módulos de la lógica de negocio (sección 3.2), de modo que cada capa tiene una responsabilidad clara: la capa de presentación solo muestra

información y captura datos; la capa de lógica de negocio aplica las reglas del dominio; y la capa de datos se limita a la persistencia.

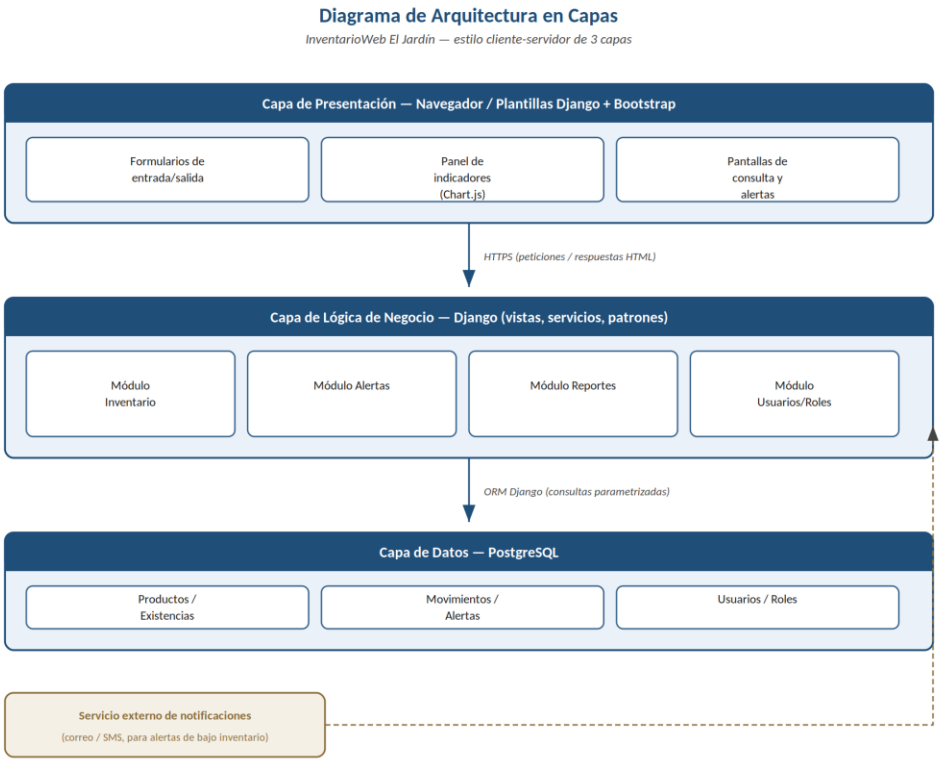


Figura 9 — Diagrama de arquitectura en capas de InventarioWeb El Jardín.

Decisiones de tecnología

La siguiente tabla resume la pila tecnológica seleccionada. Respecto de la versión anterior, se cierran las dos decisiones que habían quedado abiertas (hospedaje y canal de notificación), en línea con el objetivo de que el diseño sea lo suficientemente específico para que otro desarrollador pueda implementarlo sin tomar decisiones de arquitectura adicionales:

Capa / Aspecto	Tecnología elegida	Justificación
Backend / lógica de negocio	Django (Python) 5.x	Framework maduro, orientado a baterías incluidas: ORM, autenticación, panel de administración y sistema de permisos ya integrados, lo que reduce el código propio que un solo desarrollador debe mantener.
Frontend / presentación	Plantillas de Django + Bootstrap 5 + Chart.js	Renderizado del lado del servidor: un único proyecto desplegable, sin la complejidad

Capa / Aspecto	Tecnología elegida	Justificación
		operativa de mantener una API separada de un frontend SPA. Bootstrap agiliza una interfaz responsiva; Chart.js cubre los paneles de indicadores (RF-08, RF-17).
Base de datos	PostgreSQL 16	Motor relacional robusto y gratuito, con buen soporte de integridad referencial (clave para separar existencias por bodega/sucursal) y de funciones de agregación para los reportes comparativos.
Servidor de aplicación	Gunicorn + Nginx	Combinación estándar y de bajo costo para servir aplicaciones Django en producción, adecuada para el volumen de tráfico de un restaurante de dos sucursales.
Hospedaje	Render (plan económico)	Decisión cerrada para esta fase (antes se dejaba abierta entre Render o Railway): Render ofrece despliegue directo desde repositorio Git y un nivel de precio adecuado al presupuesto del restaurante, sin decisiones de arquitectura pendientes para el desarrollador que continúe el proyecto.
Notificaciones	SMTP (correo electrónico) en la primera versión	Se cierra la decisión de canal de notificación: la primera versión usa correo mediante SMTP. El patrón Adapter (sección 3.4) deja preparada la incorporación de SMS como un adaptador adicional sin modificar el módulo de Alertas.

Diagrama de despliegue

El diagrama muestra los nodos sobre los que se ejecutará el sistema: el dispositivo cliente, el servidor de aplicación (Django detrás de Gunicorn y Nginx) y el servidor de base de datos (PostgreSQL), además del servicio externo de notificaciones.

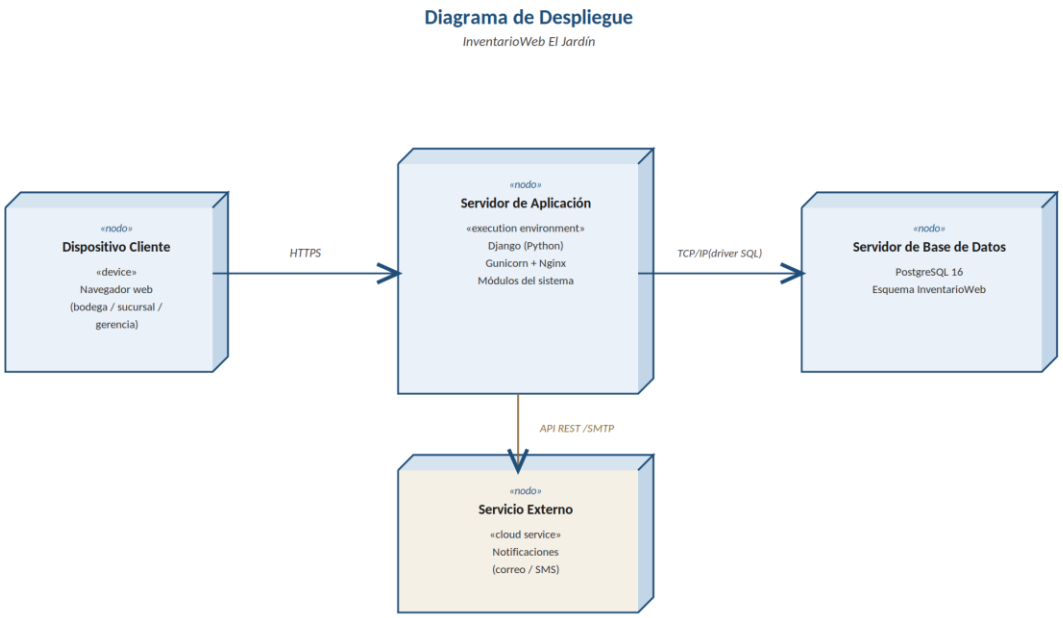


Figura 10 — Diagrama de despliegue de InventarioWeb El Jardín.

3.2 Diseño de componentes

Módulos identificados y responsabilidades

A partir de la capa de lógica de negocio se identifican seis módulos con responsabilidades bien delimitadas. Respecto de la Fase 3 original, se agrega el Módulo de Proveedores para cerrar el pendiente identificado desde la Fase 2 (clase Proveedor sin caso de uso de gestión):

Módulo	Responsabilidad	RF relacionados
Módulo de Autenticación	Gestiona el inicio y cierre de sesión, y valida que cada usuario tenga permisos para la acción solicitada (RBAC).	RF-01, RF-18
Módulo de Inventario	Registra entradas y salidas de mercadería y mantiene actualizadas las existencias por bodega y sucursal; actúa como sujeto del patrón Observer.	RF-02, RF-03, RF-04, RF-11, RF-12, RF-13, RF-15
Módulo de Alertas	Observador de Existencia: detecta cuándo una existencia cae bajo su stock mínimo y genera/gestiona las alertas correspondientes.	RF-06, RF-16
Módulo de Reportes	Observador de Existencia: calcula indicadores de consumo y alimenta el Panel de Indicadores y las exportaciones.	RF-07, RF-08, RF-14, RF-17
Módulo de Usuarios y Roles	Administra las cuentas de usuario, sus roles y el catálogo de productos.	RF-09, RF-10

Módulo	Responsabilidad	RF relacionados
Módulo de Proveedores	Administra el CRUD de proveedores (alta, edición, activar/desactivar) y expone la interfaz IProveedores que consume el Módulo de Inventario al registrar una entrada.	RF-11

Diagrama de componentes

El diagrama corrige una inconsistencia señalada en la retroalimentación: la versión anterior mostraba una relación «usa» directa entre el Módulo de Inventario y el Módulo de Alertas, lo que contradecía el argumento de desacoplamiento del patrón Observer descrito en la sección 3.4. En esta versión, Alertas y Reportes no dependen directamente de Inventario: reciben los cambios de la entidad Existencia mediante notificaciones del patrón Observer (líneas discontinuas «notifica»), sin que Inventario conozca a sus observadores. Se agrega también el Módulo de Proveedores con su interfaz IProveedores, y se completa la conexión de Reportes hacia el Panel de Indicadores.

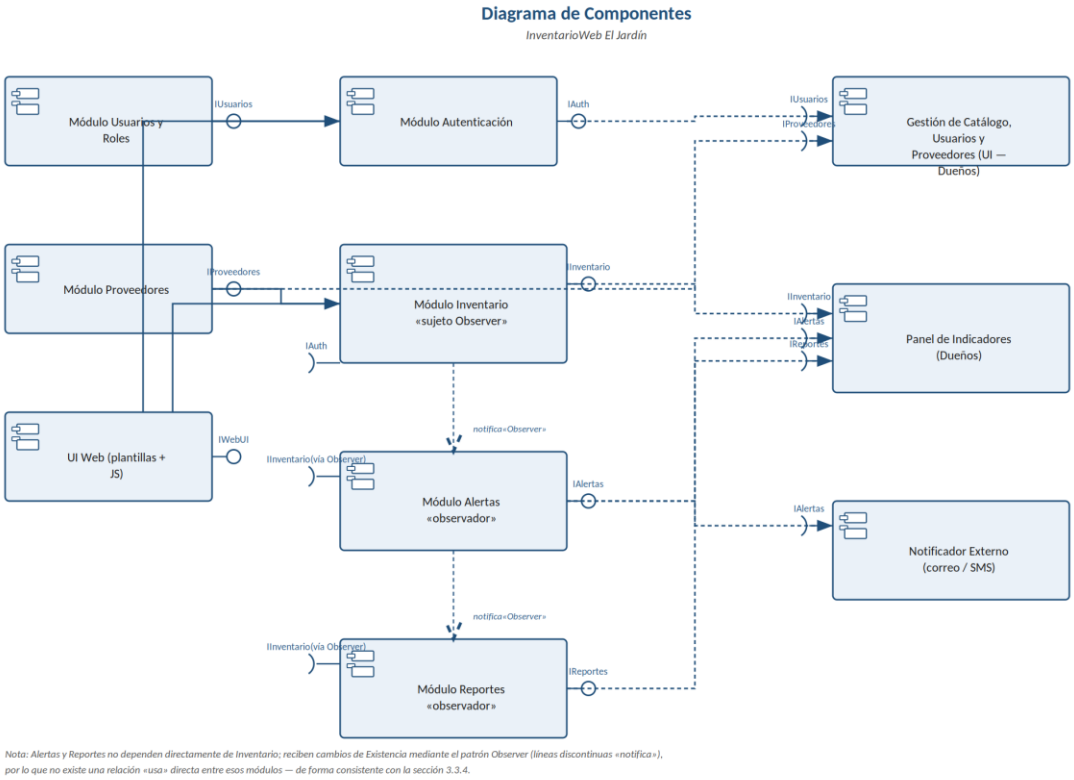


Figura 11 — Diagrama de componentes de InventarioWeb El Jardín (v1.1, corregido).

Especificación de interfaces de los componentes principales

La siguiente tabla detalla los métodos más relevantes expuestos por los componentes de la capa de lógica de negocio, incluyendo las interfaces IProveedores e IReportes agregadas en esta corrección:

Componente / Interfaz	Método	Parámetros	Retorno
IInventario	registrar_entrada	producto_id, cantidad, proveedor_id, usuario_id	MovimientoInventario
IInventario	registrar_salida	producto_id, cantidad, sucursal_id, usuario_id	MovimientoInventario
IInventario	consultar_existencias	ubicacion_id, filtros(opcional)	Lista<Existencia>
IAlertas	generar_alerta	existencia_id	AlertaInventario
IAlertas	marcar_atendida	alerta_id, usuario_id	boolean
IUsuarios	autenticar	correo, password	Usuario | error
IUsuarios	crear_usuario	datos_usuario, rol_id	Usuario
IProveedores	crear_proveedor	datos_proveedor	Proveedor
IProveedores	desactivar_proveedor	proveedor_id	boolean
IReportes	generar_indicadores	filtros(sucursal, producto, rango_fechas)	PanelIndicadores

Diagrama de clases de diseño

El diagrama de clases de análisis del Capítulo 2 se evoluciona aquí a nivel de diseño, incorporando tipos de datos concretos compatibles con el ORM de Django, claves primarias y foráneas, y la relación de herencia entre MovimientoInventario y sus subclases EntradaInventario y SalidaInventario (patrón Factory Method, sección 3.4). Se corrige además la inconsistencia señalada por el profesor: los métodos de la clase Existencia (agregar_observador(), notificar_observadores(), actualizar_cantidad(delta)) ahora coinciden exactamente entre este diagrama y el diagrama del patrón Observer de la sección 3.4.

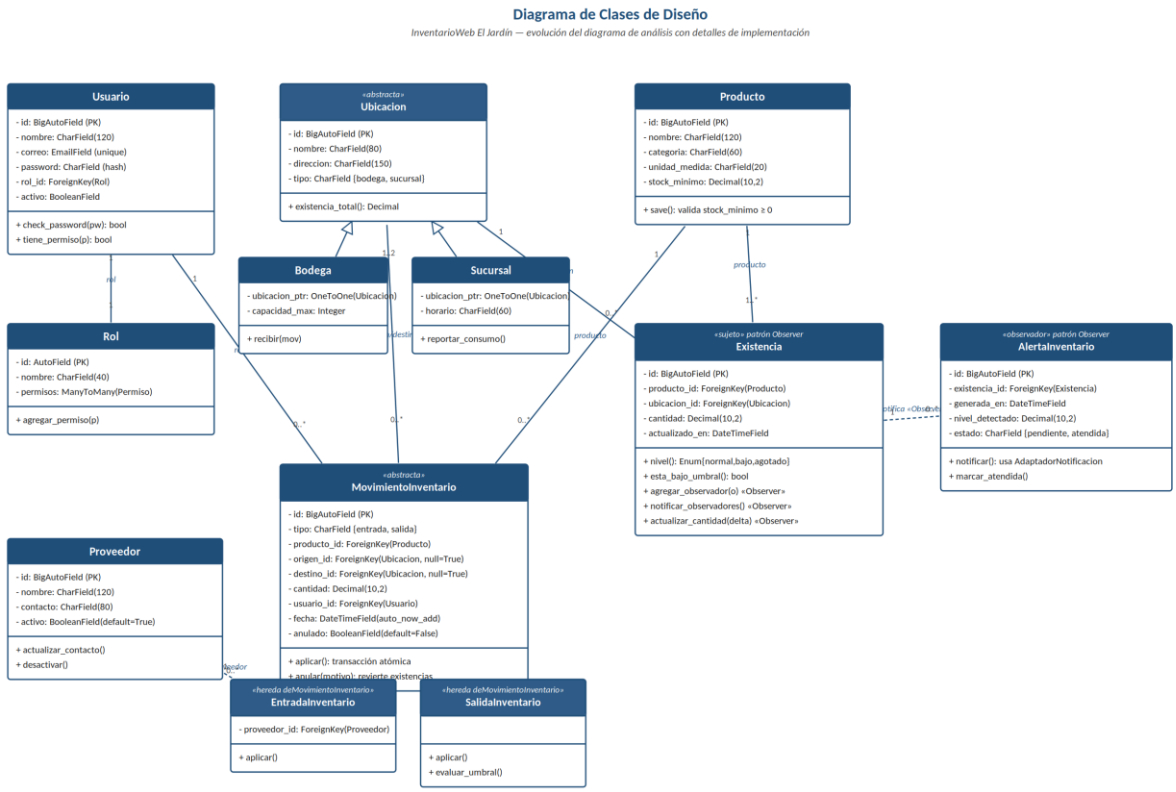


Figura 12 — Diagrama de clases de diseño de InventarioWeb El Jardín (v1.1, métodos de Existencia unificados con el patrón Observer).

Nota sobre las dos decisiones de diseño de datos pendientes desde la Fase 2, confirmadas correctas por el profesor y documentadas aquí para la presentación oral: (1) Ubicacion se modela como clase abstracta con Bodega y Sucursal como subtipos mediante herencia de tabla (OneToOne), el patrón estándar de Django para este caso; (2) Existencia se modela como entidad persistida (con cantidad y actualizado_en), no calculada dinámicamente sumando movimientos en cada consulta, decisión que prioriza RNF-01 (respuestas menores a 2 segundos).

3.3 Diseño de la interfaz de usuario

Se presentan siete wireframes de las pantallas principales del sistema (las seis originales, sin cambios, más la nueva pantalla de gestión de proveedores), seguidos del prototipo de navegación, la guía de estilo visual y los principios de usabilidad aplicados.

Wireframes de las pantallas principales

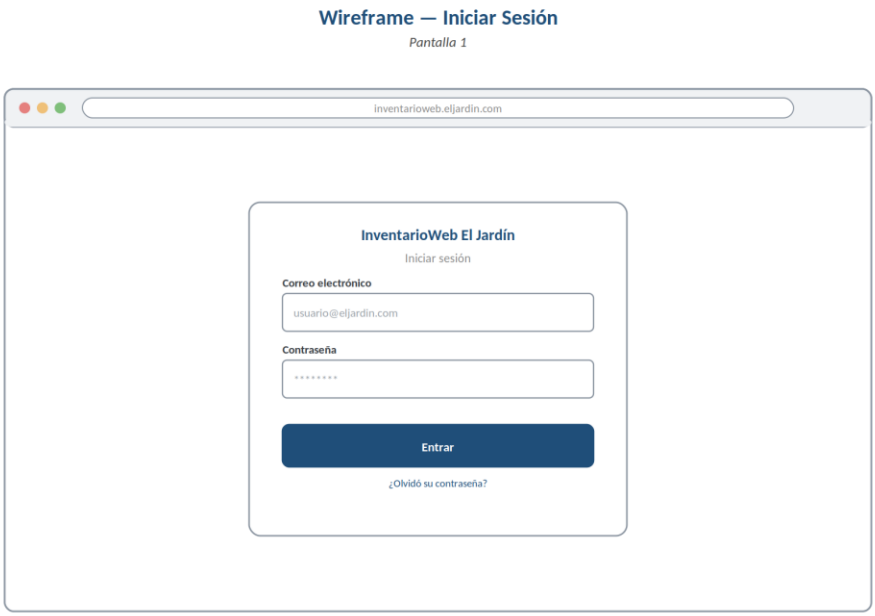


Figura 13 — Pantalla de inicio de sesión, común a los tres roles.

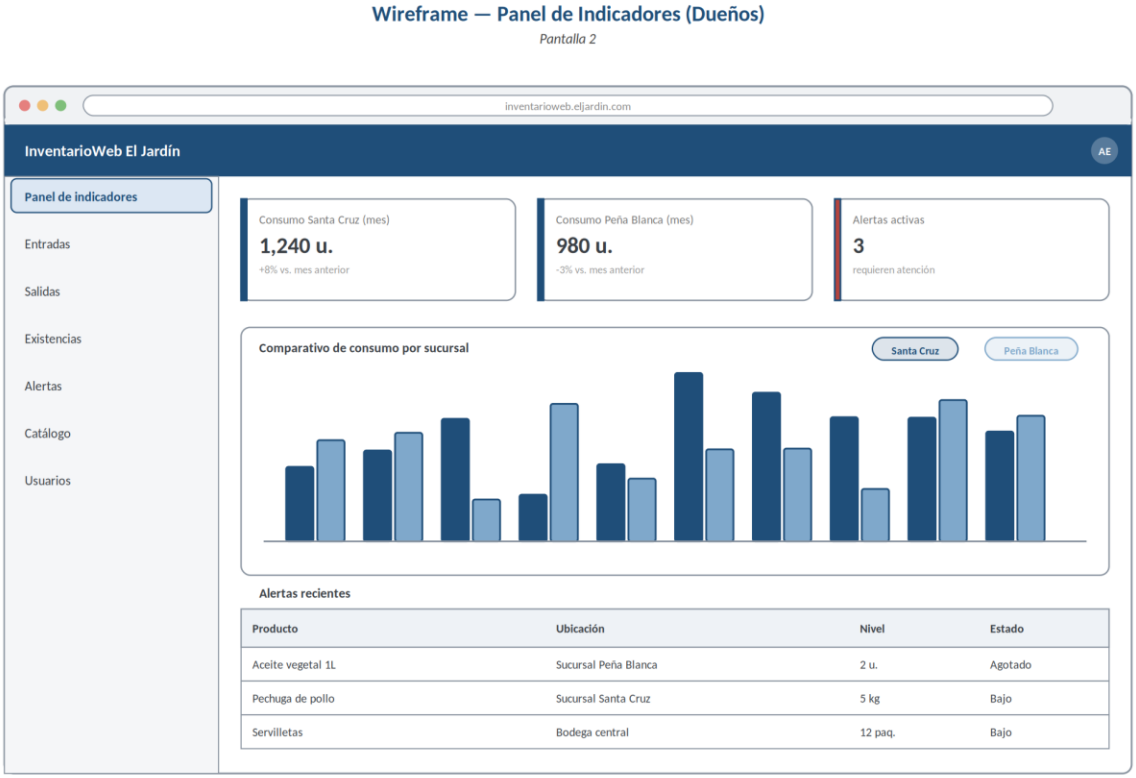


Figura 14 — Panel de indicadores comparativo entre sucursales, para los dueños (UC07).

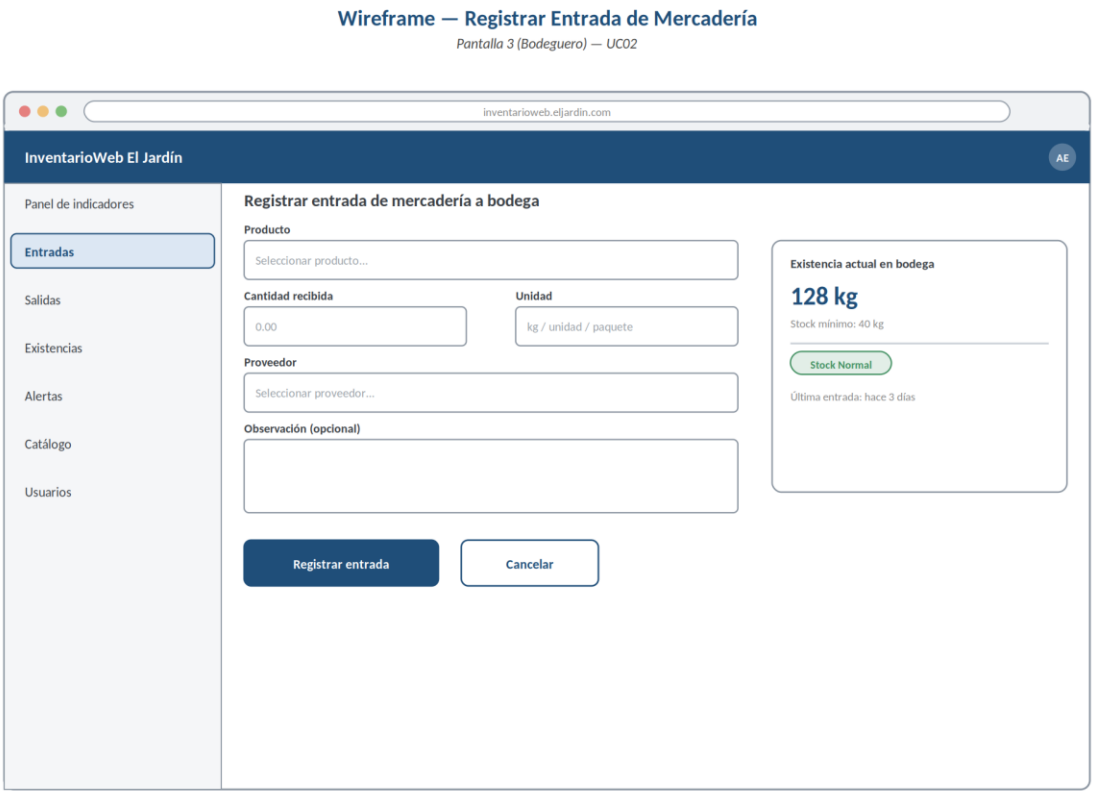


Figura 15 — Registro de entrada de mercadería a bodega, con la existencia actual visible como contexto (UC02).

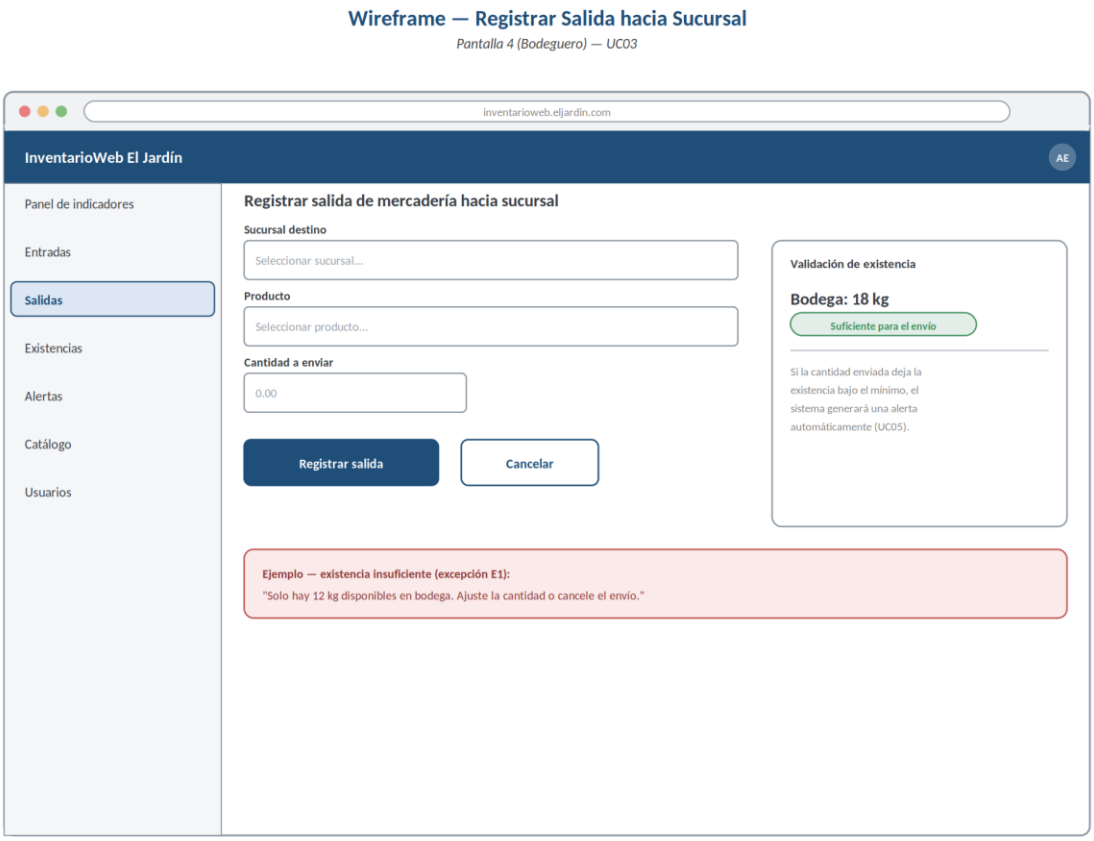


Figura 16 — Registro de salida hacia sucursal, con validación de existencia y aviso de excepción (UC03).

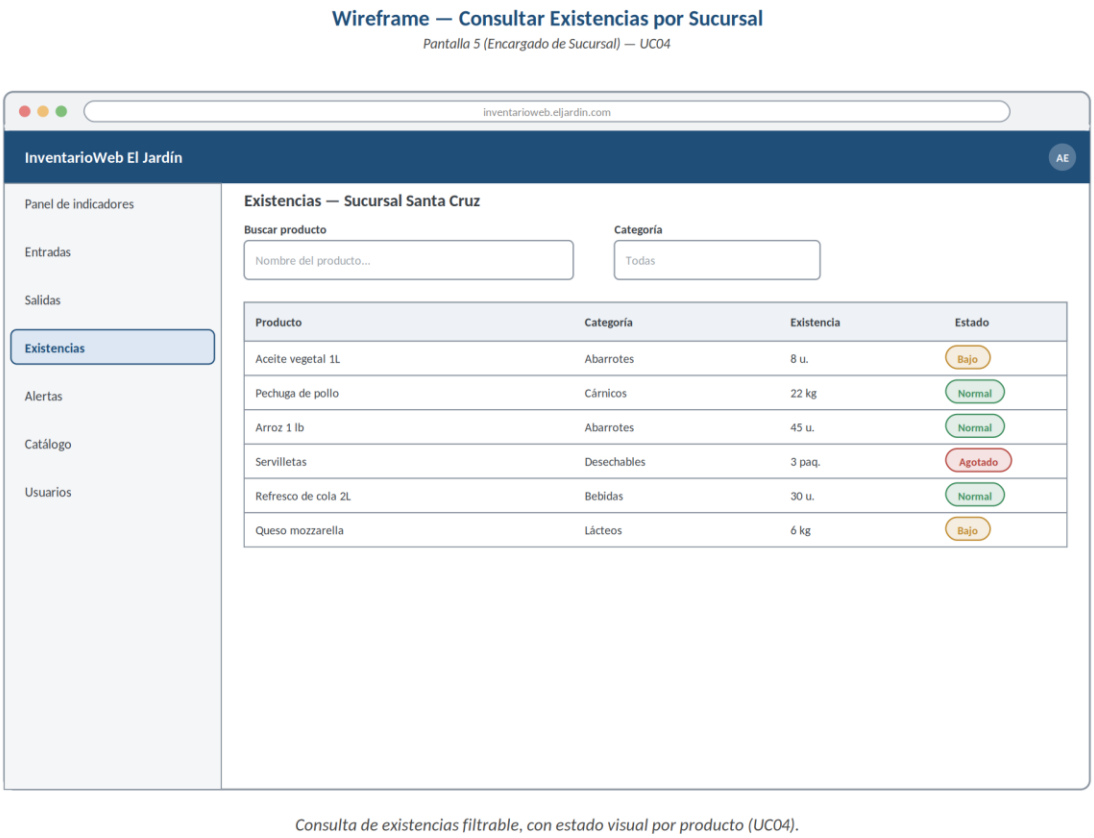


Figura 17 — Consulta de existencias por sucursal, filtrable y con estado visual por producto (UC04).

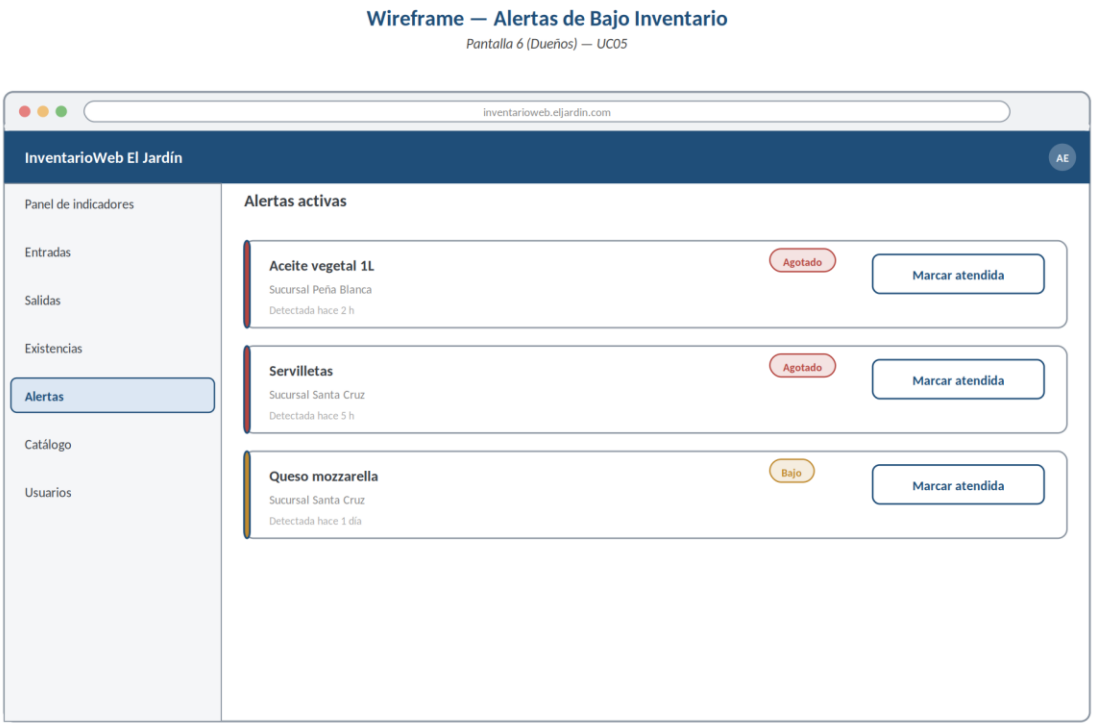


Figura 18 — Listado de alertas de bajo inventario pendientes (UC05).

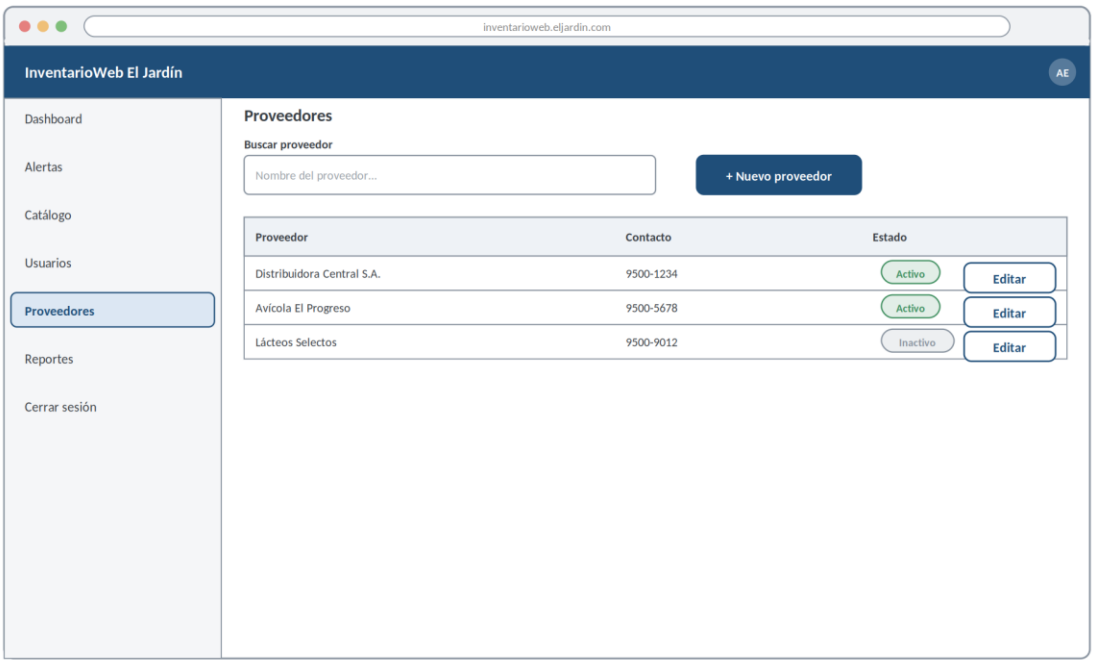


Figura 19 — Gestión de proveedores (CRUD básico) — pantalla nueva que cierra el pendiente de la Fase 2.

Prototipo de navegación

El siguiente diagrama muestra el flujo típico de pantallas para cada rol a partir del inicio de sesión. El menú lateral, persistente en toda la aplicación, permite moverse libremente entre las pantallas habilitadas para el rol activo.

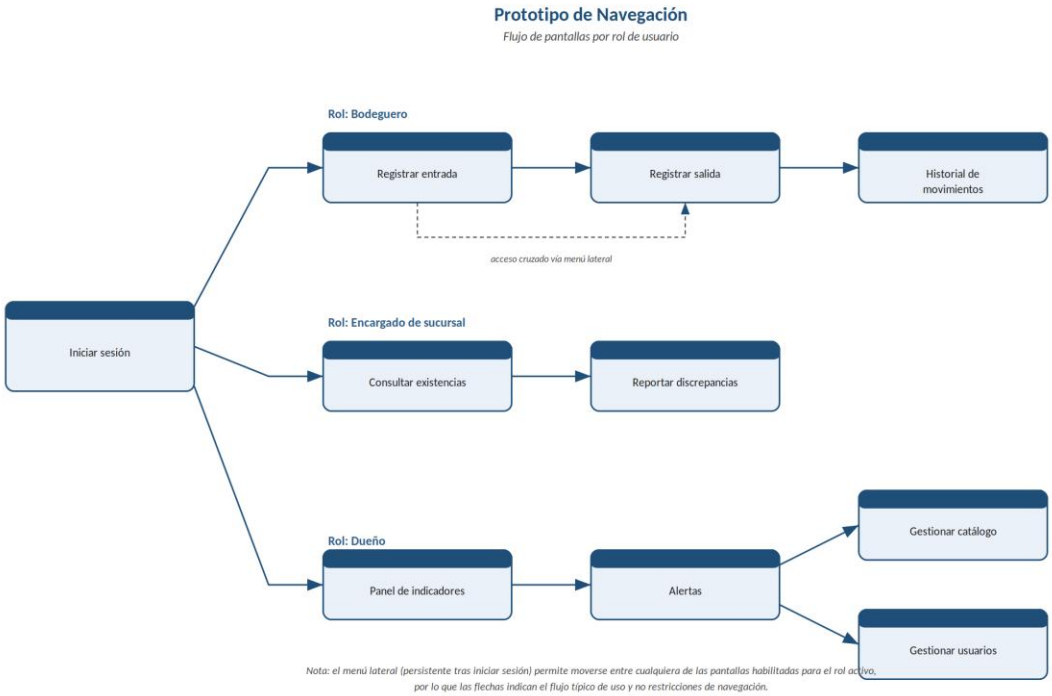


Figura 20 — Prototipo de navegación de InventarioWeb El Jardín por rol de usuario.

Guía de estilo

Se define una guía de estilo simple y consistente, con un criterio de accesibilidad explícito: el estado de una existencia siempre se representa con color y texto en una "pill" (nunca solo con color).

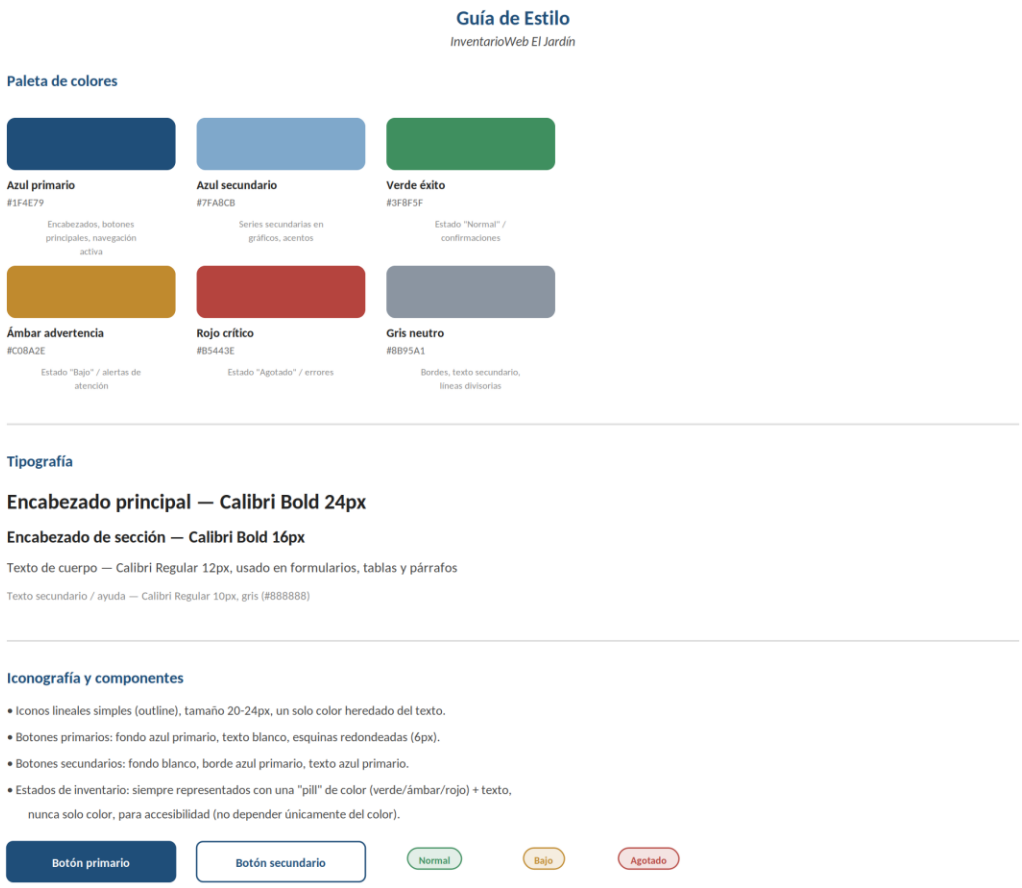


Figura 21 — Guía de estilo de InventarioWeb El Jardín: paleta de colores, tipografía y componentes base.

Principios de usabilidad aplicados

- Visibilidad del estado del sistema: toda pantalla muestra de inmediato la existencia actual o el resultado de la acción.
- Prevención de errores: el formulario de salida valida la existencia disponible antes de enviar el registro y sugiere la cantidad máxima disponible en vez de solo rechazar la operación.
- Reconocer antes que recordar: el menú lateral permanece visible en todo momento.
- Consistencia y estándares: los mismos componentes visuales (pills de estado, botones primarios/secundarios) se repiten en todas las pantallas, incluida la nueva de proveedores.

- Diseño minimalista: cada formulario solicita solo los campos estrictamente necesarios (RNF-03: tareas frecuentes en máximo 3 pasos).

3.4 Diseño basado en patrones

Pressman (cap. 12, con base en Gamma et al.) clasifica los patrones de diseño en creacionales, estructurales y conductuales. Se identifican tres patrones, uno de cada categoría, cada uno resolviendo un problema concreto ya identificado en fases anteriores.

Patrón 1 — Factory Method (creacional)

Problema que resuelve: el sistema debe crear instancias de EntradaInventario o SalidaInventario según el caso de uso, sin que el código de las vistas necesite conocer el detalle de cada subclase concreta. MovimientoInventarioFactory centraliza esa decisión.

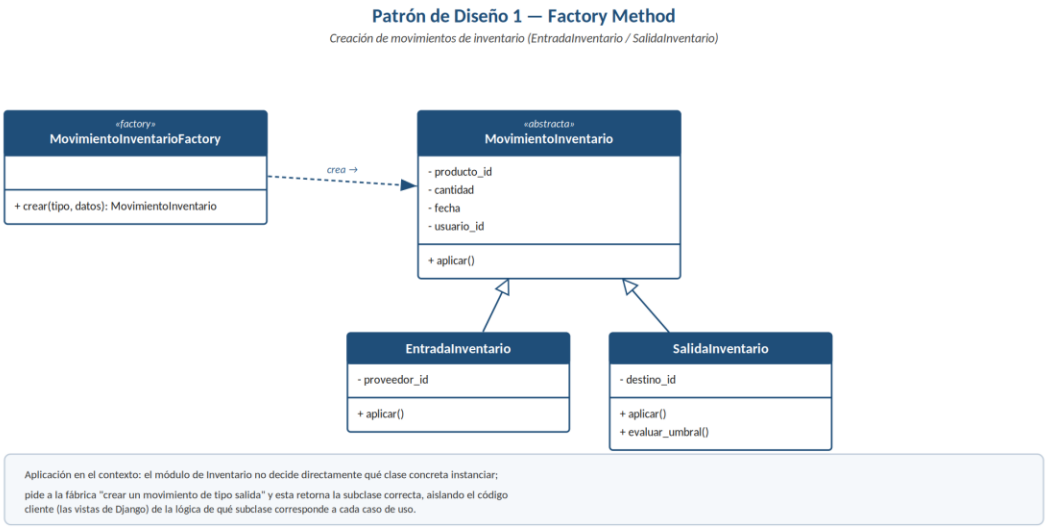


Figura 22 — Aplicación del patrón Factory Method a la creación de movimientos de inventario.

Patrón 2 — Observer (conductual)

Problema que resuelve: cuando una Existencia cambia de estado, dos módulos distintos deben reaccionar de forma independiente: Alertas (para generar una alerta) y Reportes (para reflejar el nuevo dato en el panel). El patrón Observer permite que Existencia notifique a ambos sin conocer sus detalles internos. Esta es la misma relación que ahora se refleja consistentemente en el diagrama de componentes corregido (sección 3.2) mediante las líneas «notifica».

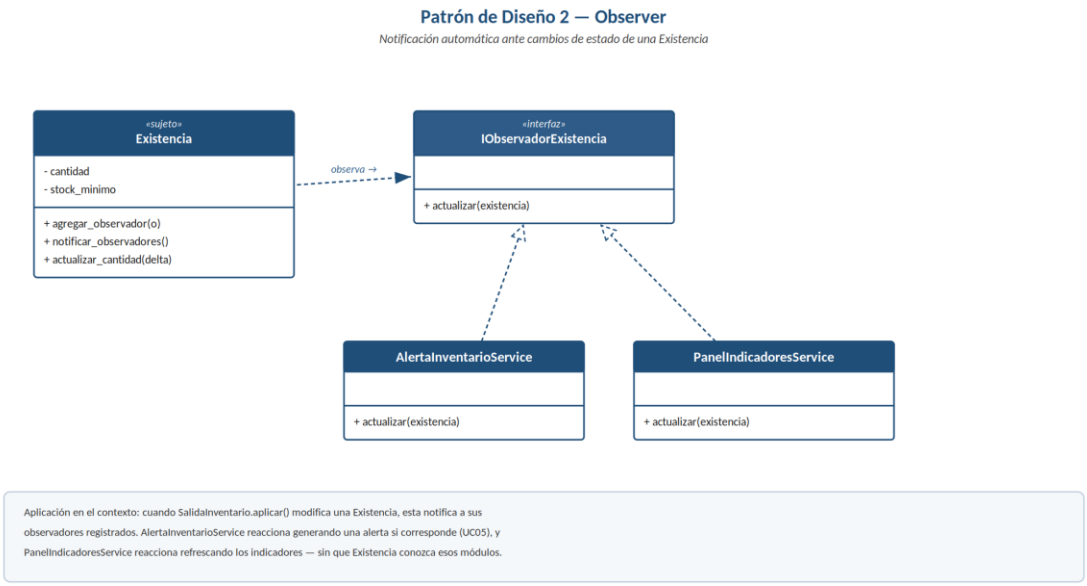


Figura 23 — Aplicación del patrón Observer a la notificación de cambios de existencia.

Patrón 3 — Adapter (estructural)

Problema que resuelve: el módulo de Alertas notifica a los dueños por correo (SMTP en la primera versión); el patrón Adapter introduce la interfaz común INotificador para que, si más adelante se agrega SMS, solo se sume un nuevo adaptador sin modificar el módulo de Alertas.

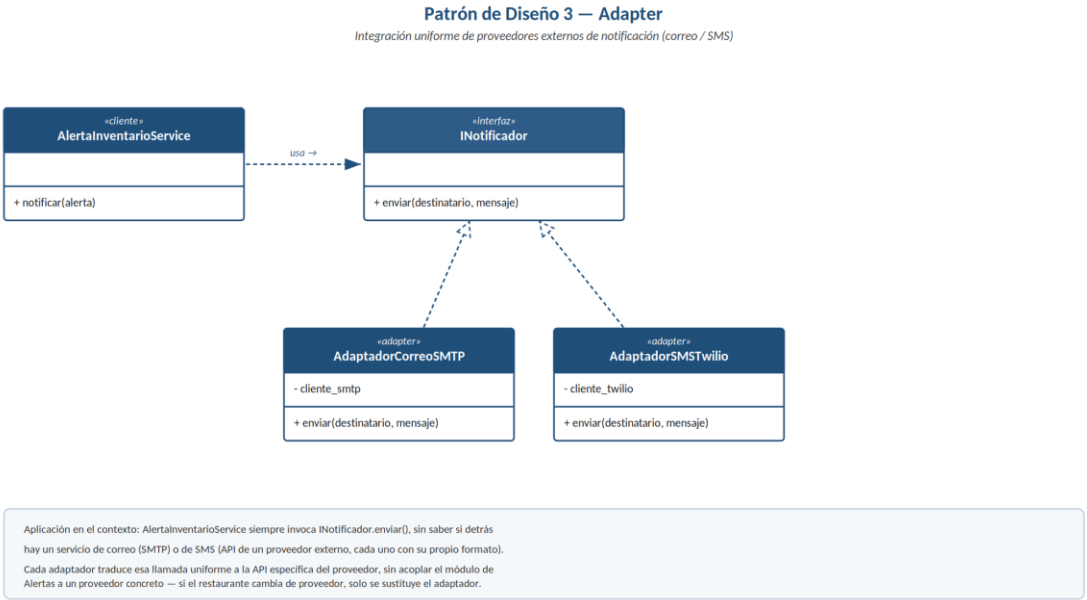


Figura 24 — Aplicación del patrón Adapter a la integración de proveedores externos de notificación.

3.5 Diseño de la webapp

Mapa de contenido (sitemap)

El mapa organiza el contenido según el rol del usuario autenticado. Respecto de la versión anterior, se agrega "Gestionar proveedores" al alcance de Dueños, cerrando el pendiente señalado por el profesor.

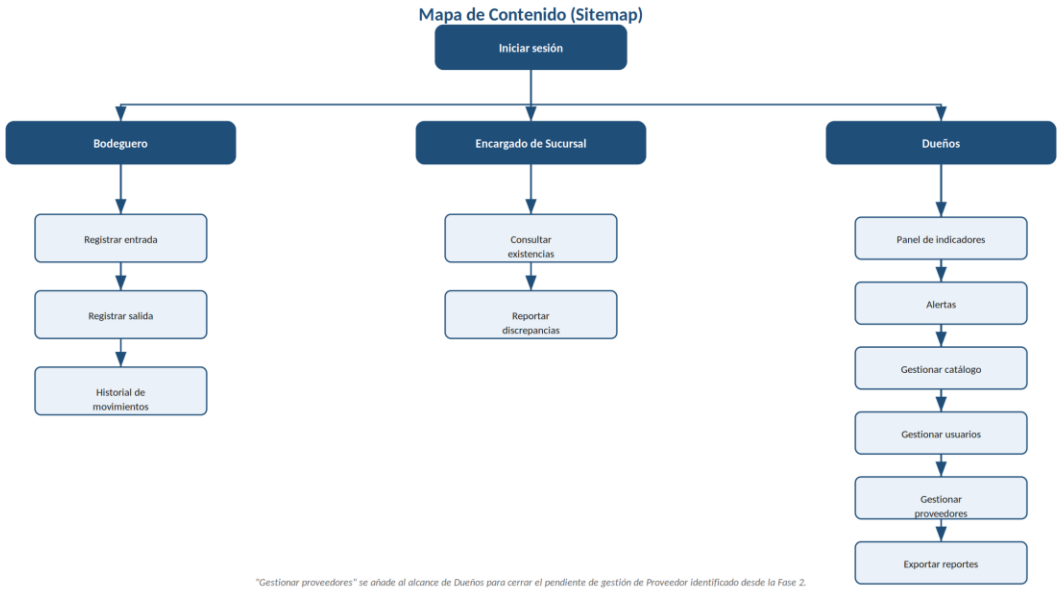


Figura 25 — Mapa de contenido de InventarioWeb El Jardín (v1.1, con Gestionar proveedores).

Diseño de la navegación

El modelo de navegación es jerárquico con acceso lateral persistente: tras autenticarse, el usuario ve un menú fijo con únicamente las secciones habilitadas para su rol, evitando exponer funciones que no le corresponden.

Arquitectura de la información

El contenido se organiza en torno a tres ejes: operación diaria (entradas y salidas, para el bodeguero), consulta local (existencias y discrepancias, para encargados de sucursal) y supervisión (indicadores, alertas, catálogo, usuarios, proveedores y reportes, para los dueños).

Consideraciones de seguridad

- Autenticación y control de acceso por rol (RBAC) mediante el sistema de permisos de Django (RF-01, RNF-02).

- Contraseñas almacenadas con función de hash; comunicación sobre HTTPS; protección CSRF y ORM parametrizado contra inyección SQL.
- Gestión de sesión: expiración automática por inactividad y cierre seguro con invalidación del token de sesión tras logout.
- Protección de datos sensibles: credenciales de servicios externos (SMTP) gestionadas mediante variables de entorno, nunca en el código fuente.
- Control de intentos de autenticación: bloqueo temporal tras varios intentos fallidos consecutivos de inicio de sesión.
- Los encargados de sucursal solo pueden consultar y operar sobre la ubicación que tienen asignada (principio de mínimo privilegio).

Consideraciones de rendimiento y escalabilidad

- Índices de base de datos sobre las columnas más consultadas (producto_id, ubicacion_id) para sostener RNF-01.
- Consultas de indicadores agregadas a nivel de base de datos (PostgreSQL) en lugar de traer todos los movimientos al servidor de aplicación.
- Arquitectura en capas desacoplada que permite escalar horizontalmente el servidor de aplicación si el restaurante añadiera nuevas sucursales.
- La separación de responsabilidades mediante los patrones Observer y Adapter permite añadir nuevos canales de notificación (por ejemplo, WhatsApp) sin modificar la lógica de negocio ni el módulo de Inventario.

Capítulo 4 — Reflexiones y Lecciones Aprendidas

4.1 Decisiones técnicas clave

La otra decisión que me costó más de lo que esperaba fue de modelado de datos: hacer que Ubicacion sea una clase abstracta de la que heredan Bodega y Sucursal, en vez de simplemente ponerle una columna "tipo" a una tabla genérica y ya. Esto viene directo del problema que veníamos arrastrando desde la Fase 1 — separar el inventario por sucursal sin duplicar lógica —, y Django tiene una forma nativa de resolverlo con herencia de tabla (OneToOne). De ahí salió también la decisión de guardar la Existencia como su propio dato, en vez de calcularla cada vez sumando movimientos. Ahí elegí rendimiento sobre elegancia: sí, es más trabajo mantener esa cifra consistente, pero el bodeguero y los encargados de sucursal son quienes más van a usar el sistema día a día, y no les puedo pedir que esperen a que el sistema sume cien movimientos cada vez que abren la pantalla de existencias.

4.2 Dificultades encontradas y soluciones aplicadas

Lo que pasó fue simple: hice el diagrama de componentes en un momento del proceso y el diseño de clases con Observer en otro, y nunca volví atrás a revisar si seguían contando lo mismo. Corregirlo no fue difícil una vez que lo vi — rehacer el diagrama para que Alertas y Reportes aparecieran como observadores, no como consumidores directos —, pero la lección de fondo sí me quedó: un documento de diseño no se revisa capítulo por capítulo, se revisa cruzando todos los diagramas entre sí, porque los errores más vergonzosos son justo los que aparecen en el cruce.

Y hubo un problema, más chiquito pero igual de revelador: la clase Proveedor llevaba desde la Fase 2 en el modelo de datos, pero nunca tuvo módulo, ni interfaz, ni pantalla propia. Quedó ahí, referenciada por EntradaInventario, como flotando sin nadie que la administrara. Cerrarlo significó agregar un sexto módulo, una interfaz IProveedores y un wireframe de CRUD básico. Me sirvió para confirmar algo que ahora aplico como regla: si una clase existe en el modelo, tiene que poder rastrearse hasta al menos una pantalla o un módulo que la gestione — si no, tarde o temprano alguien la va a notar "flotando".

4.3 Aspectos a mejorar

- Llevar desde la Fase 2 una matriz de trazabilidad RF → módulo → pantalla → clase, en vez de reconstruirla de memoria al final. Me habría ahorrado tanto el pendiente de Proveedores como la inconsistencia del diagrama de componentes.
- Cada vez que agrego una figura nueva, revisarla contra las figuras anteriores que comparten clase o módulo — así habría visto antes que los métodos de Existencia no coincidían entre el diagrama de clases de diseño y el del patrón Observer.
- Cerrar las decisiones tecnológicas la primera vez que las escribo, no dejarlas "para revisar después".

4.4 Aprendizajes para la fase de desarrollo

Lo que aprendí tiene que ver más con el proceso que con el código: entregar por fases y recibir observaciones puntuales funcionó mucho mejor que perseguir la perfección desde la primera entrega. Las correcciones que más valor le dieron al proyecto — el plan de trabajo que faltaba en la Fase 1, la consistencia del diagrama de componentes en la Fase 3 — aparecieron precisamente porque hubo una revisión externa después de cada entrega parcial, no porque yo las hubiera detectado sola encerrada revisando mi propio trabajo. Pienso hacer lo mismo en el desarrollo: entregar en pedazos chicos y revisables, en vez de guardarme todo para una gran entrega al final.

Y para cerrar, algo que cambió más de lo que esperaba mi forma de trabajar: cuando el proyecto es para un negocio real, la pregunta deja de ser "¿esto está técnicamente bien hecho?" y pasa a ser "¿voy a poder mantener esto yo sola, con el presupuesto de un restaurante pequeño, cuando ya no esté en el curso?". Esa pregunta fue la que realmente inclinó la balanza hacia Django en vez de un stack más vistoso pero más caro de operar, y es la que me va a seguir guiando cuando empiece a programar el próximo cuatrimestre.

Referencias Bibliográficas

- Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.
- IEEE Computer Society. (1998). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Django Software Foundation. (2026). Django documentation (versión 5.x). https://docs.djangoproject.com/
- PostgreSQL Global Development Group. (2026). PostgreSQL 16 documentation. https://www.postgresql.org/docs/
- Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. Resumen consultado en https://martinfowler.com/eaaCatalog/
- Render Services, Inc. (2026). Render documentation. https://render.com/docs